

CSC490 Module 1 Report

Group Members' Information

Part A

Part 2.1.2 Voxelization

Part 2.2.5 Overfitting visualization

Detections.png

Predicted_heatmap.png

Target_heatmap.png

Part 2.2.6 Training + Testing Visualizations

On Training set

Part 2.3.3 Plots + Discussions

Part 2.3.4

Part B

Motivation

What is the problem being Tackled

Why is this problem relevant

Briefly explain other approaches to this problem

Techniques

Describe the intuition for why your approach would help

Articulate your approach mathematically/algorithmically

Evaluation

Articulate your evaluation plan mathematically/algorithmically

Show compelling qualitative results/comparisons

Experiment 1: Visualizations Generated From model using just focal loss

Gamma = 2, alpha = 0.25

Gamma = 1, alpha = 0.25

Experiment 2: Visualizations Generated From model using just Anisotropic Gaussian

Scale x = 2 * (boxsize_x ^ 2); Scale y = 2 * (boxsize_y ^ 2); (original one)

Scale x = 1 * (boxsize_x ^ 2); Scale y = 1 * (boxsize_y ^ 2)

Experiment 3 Combining both anisotropic gaussian and focal loss

Show compelling quantitative results

Experiment 1 (Just focal loss)

Experiment 2 (Just Anisotropic Gaussian Kernel)

Limitation

Are there setups in which your approach fails

Are there potential paths of future work to improve your approach

Group Members' Information

Member 1

First Name: Lily Xiangning
Last Name: Yu
Student No: 1005219292
UTORID: yuxiangn

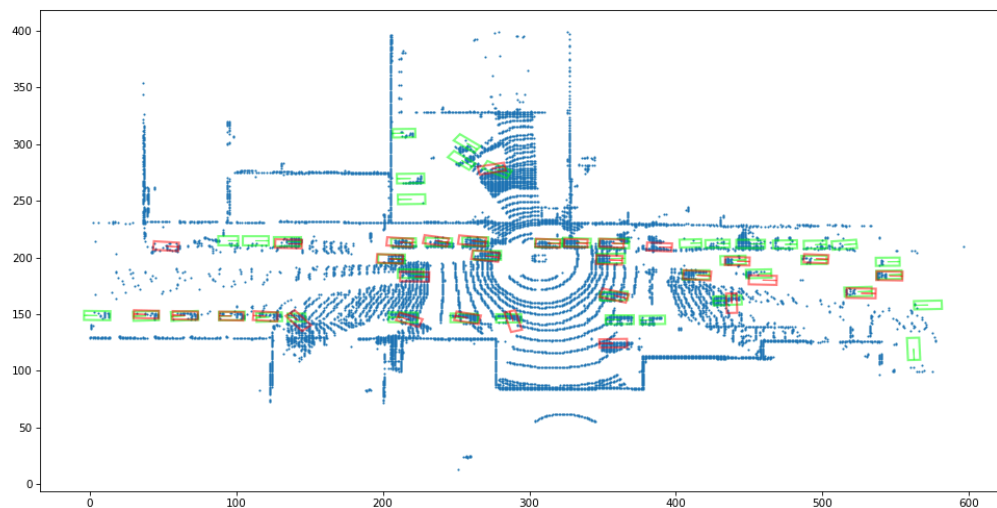
Member 2

First Name: Kwan Kiu
Last Name: Choy
Student No: 1005005879
UTORID: choykwan

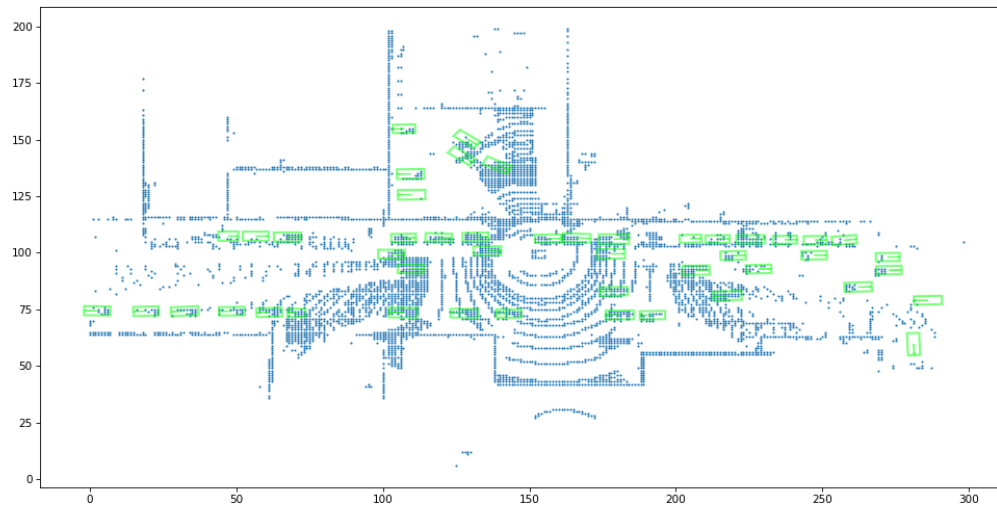
Part A

Part 2.1.2 Voxelization

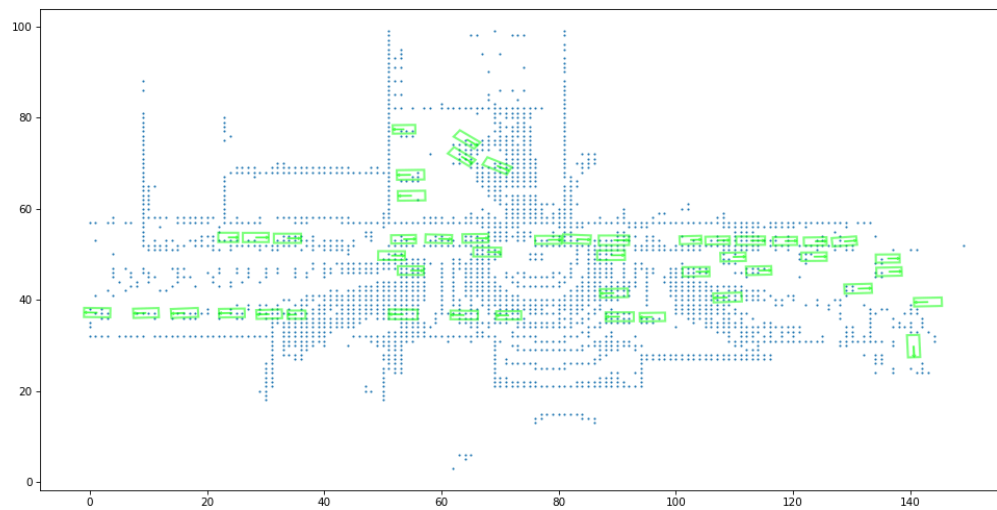
Step = 0.25



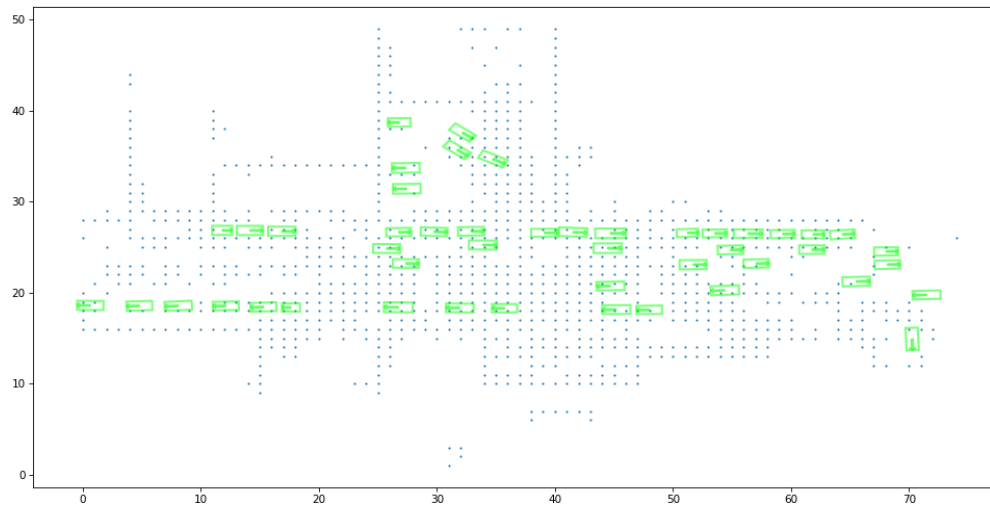
Step = 0.5



Step = 1



Step = 2

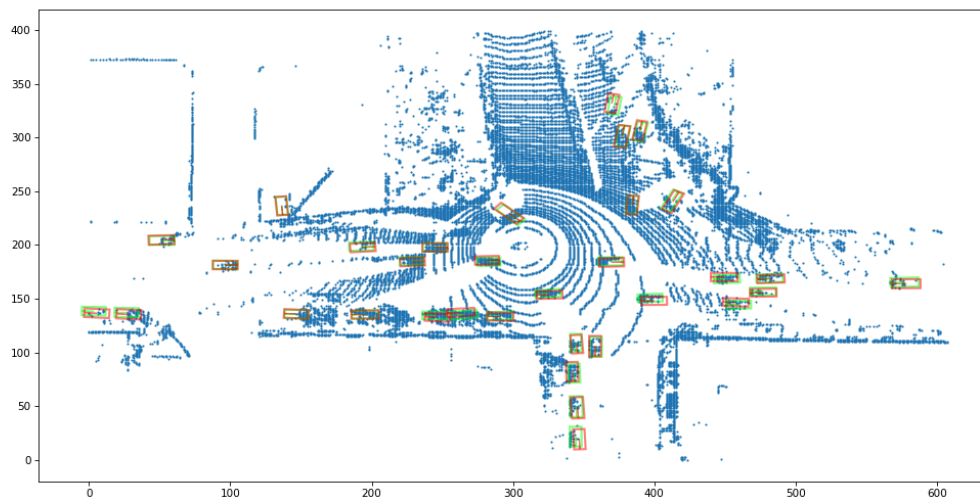


Discussion:

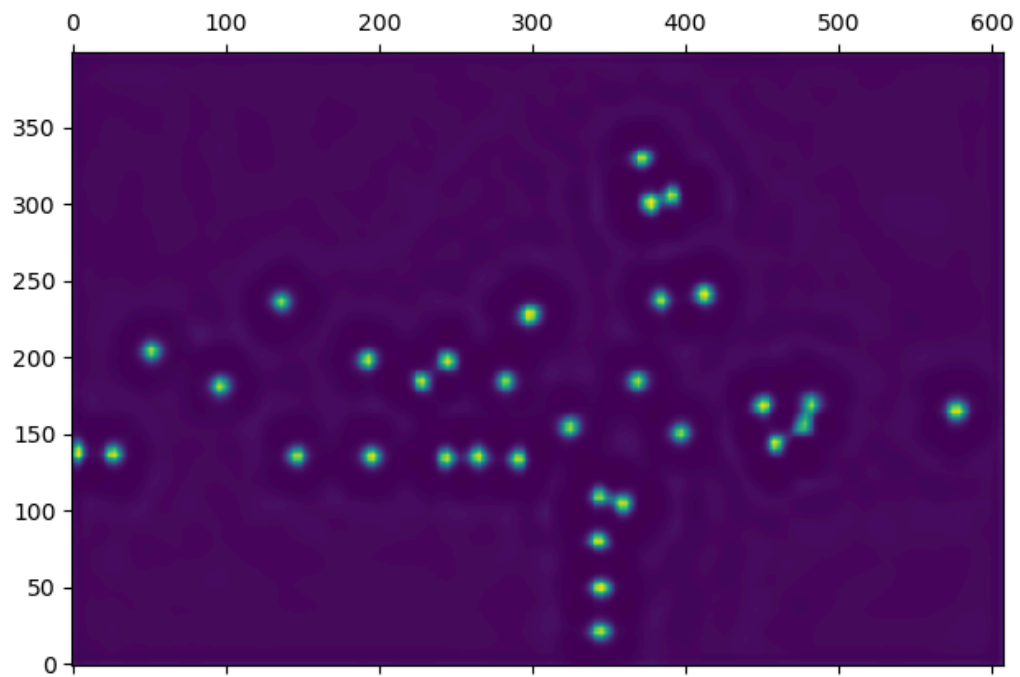
As we can see, as steps increase from 0.25 to 2, the dots become sparser and less details are captured by the blue dots. This is particularly obvious in the circular wave centered at around (310, 200). This implies a drop in fidelity as step increases. However, on the other hand, since we are capturing less blue dots in the map, the memory required will also decrease. This means that as the fidelity of the voxel grid increases, the memory consumption will also increase.

Part 2.2.5 Overfitting visualization

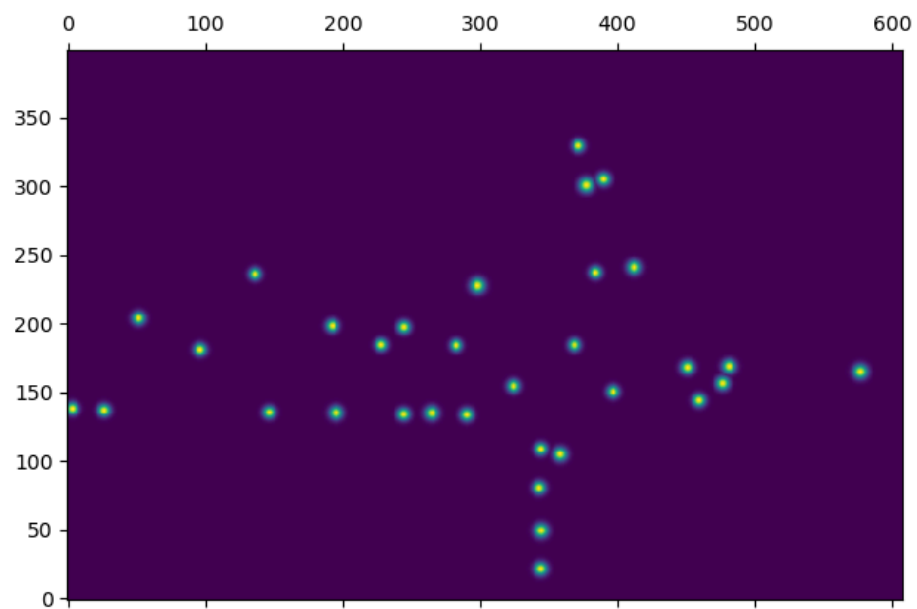
Detections.png



Predicted_heatmap.png



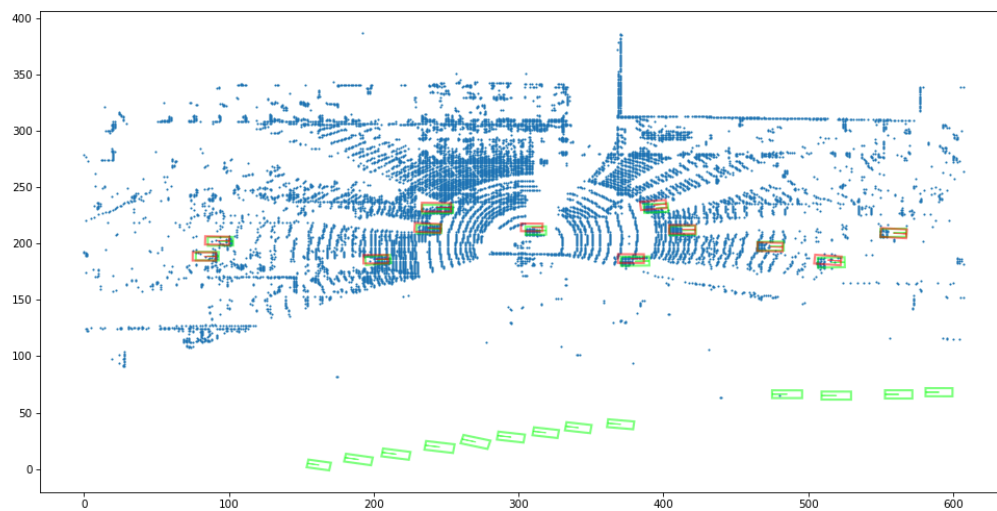
Target_heatmap.png



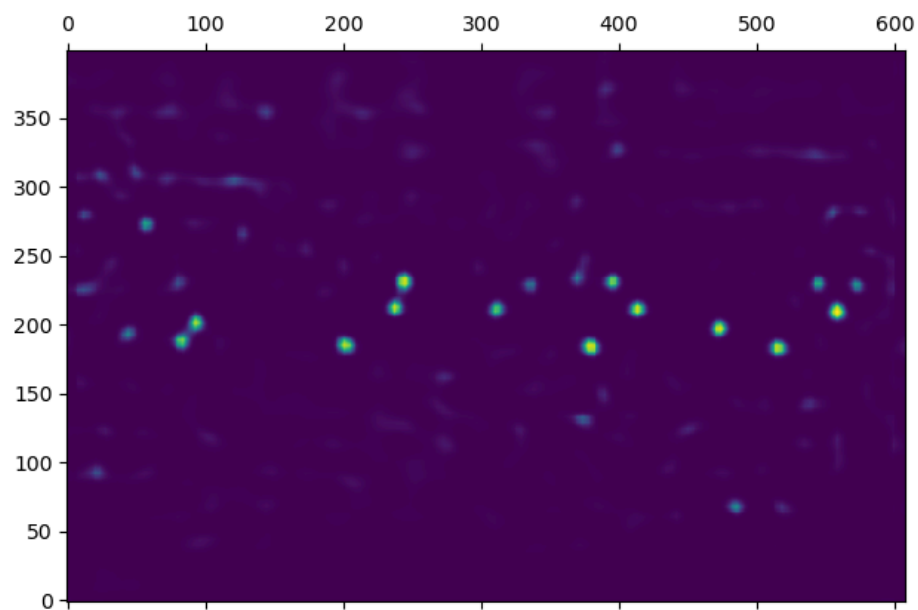
Part 2.2.6 Training + Testing Visualizations

On Training set

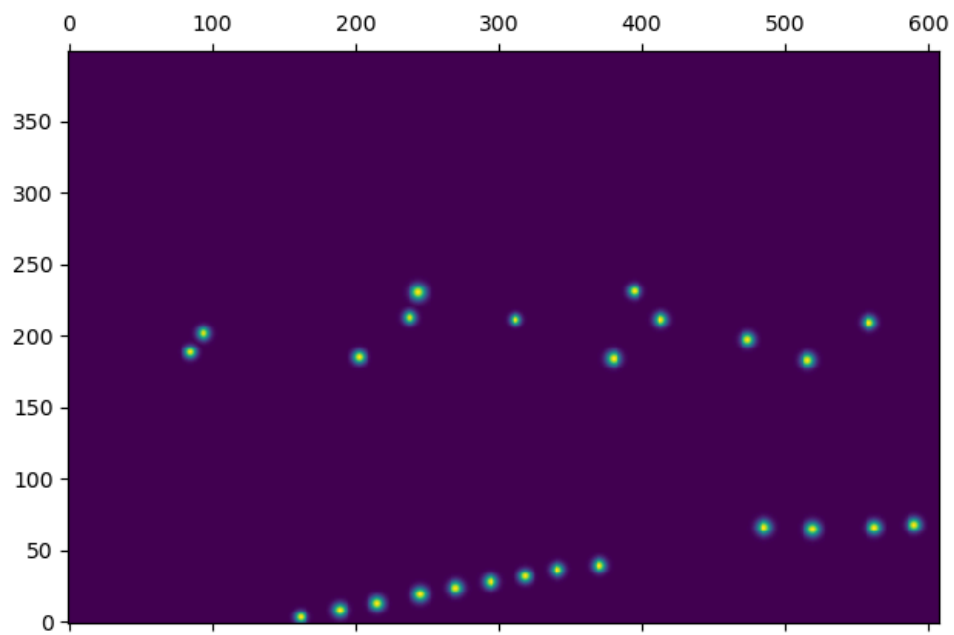
Detections.png



Predicted_heatmap.png

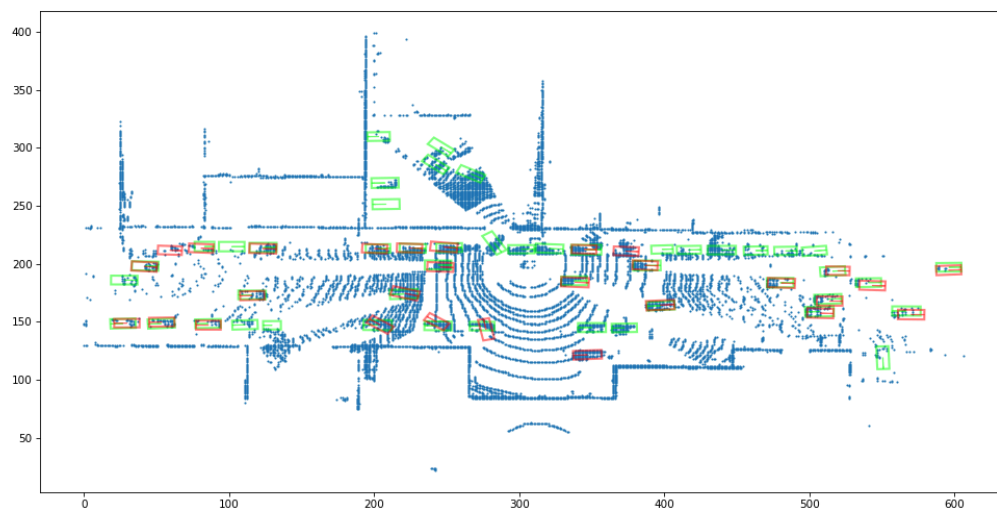


Target_heatmap.png

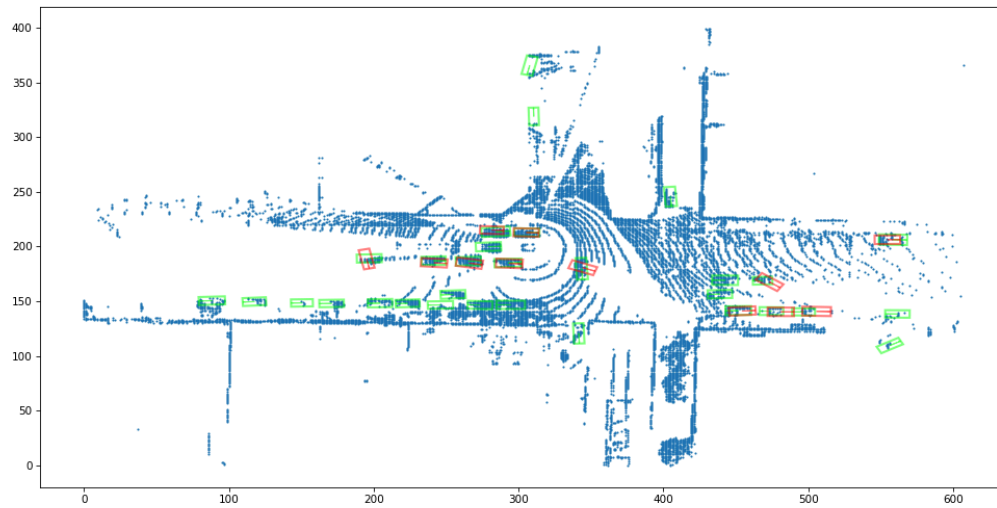


On Validation Set

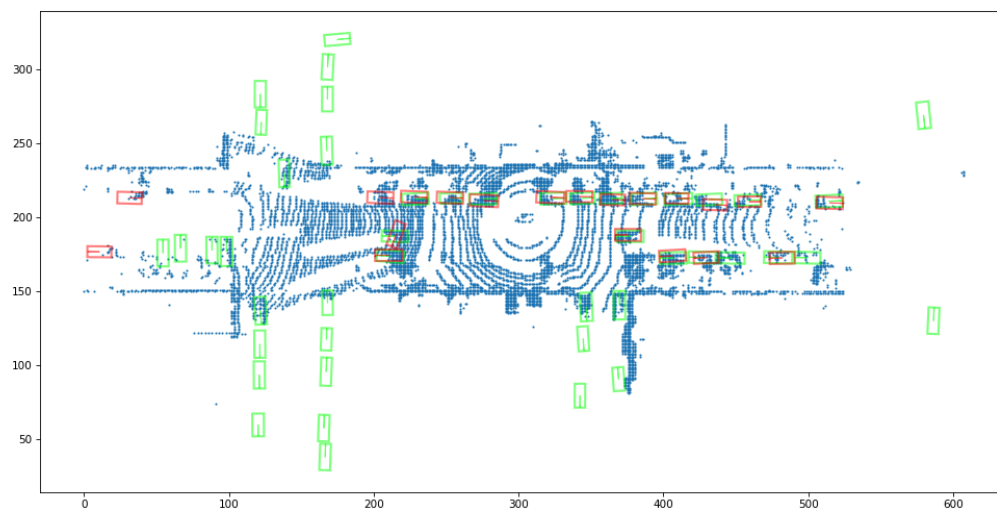
40.png:



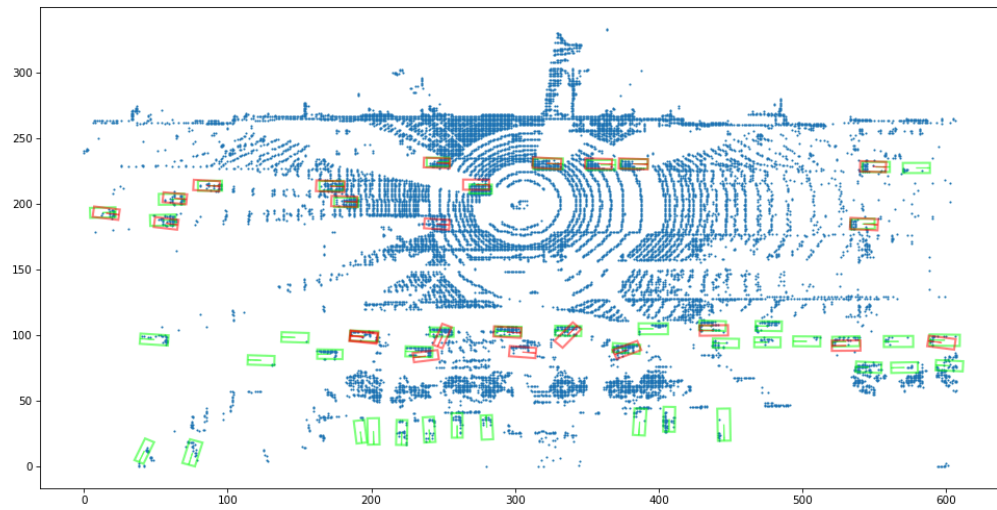
160.png:



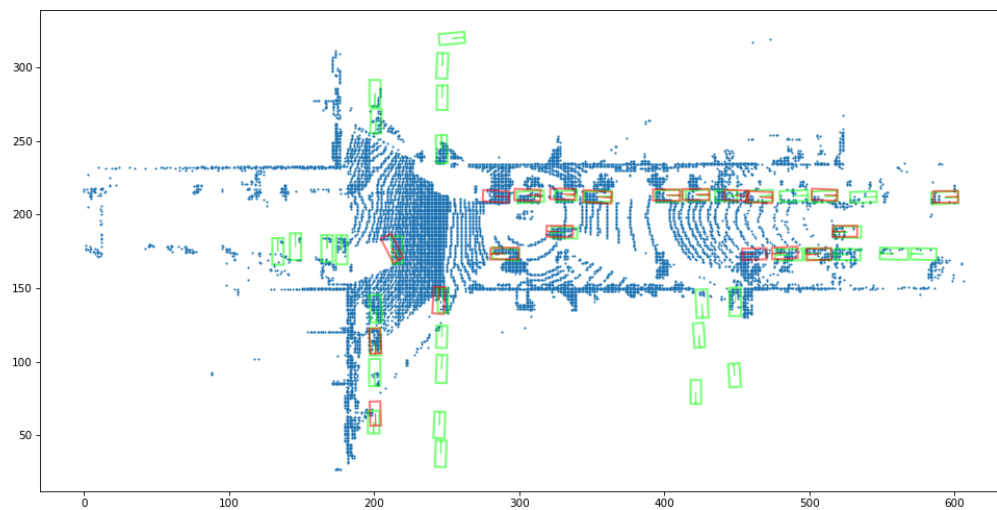
670.png:



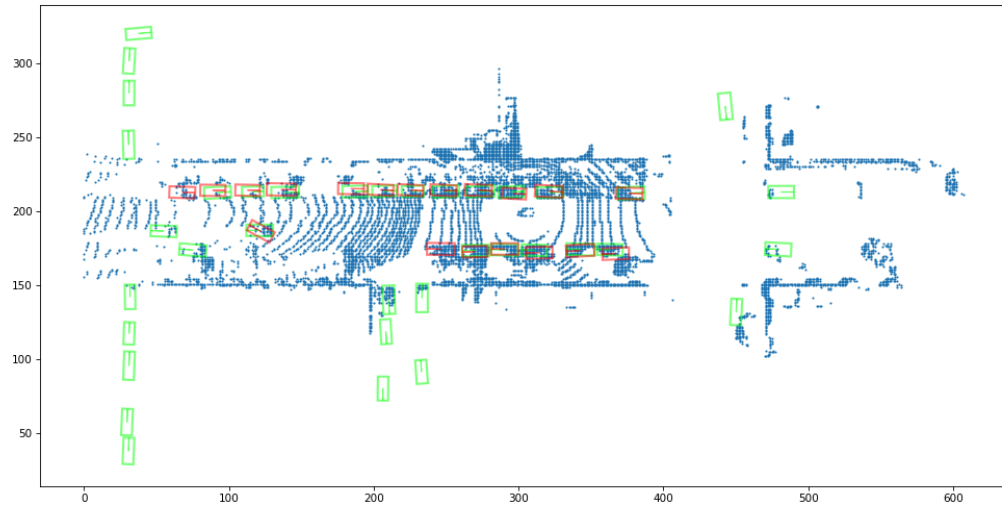
500.png:



650.png:

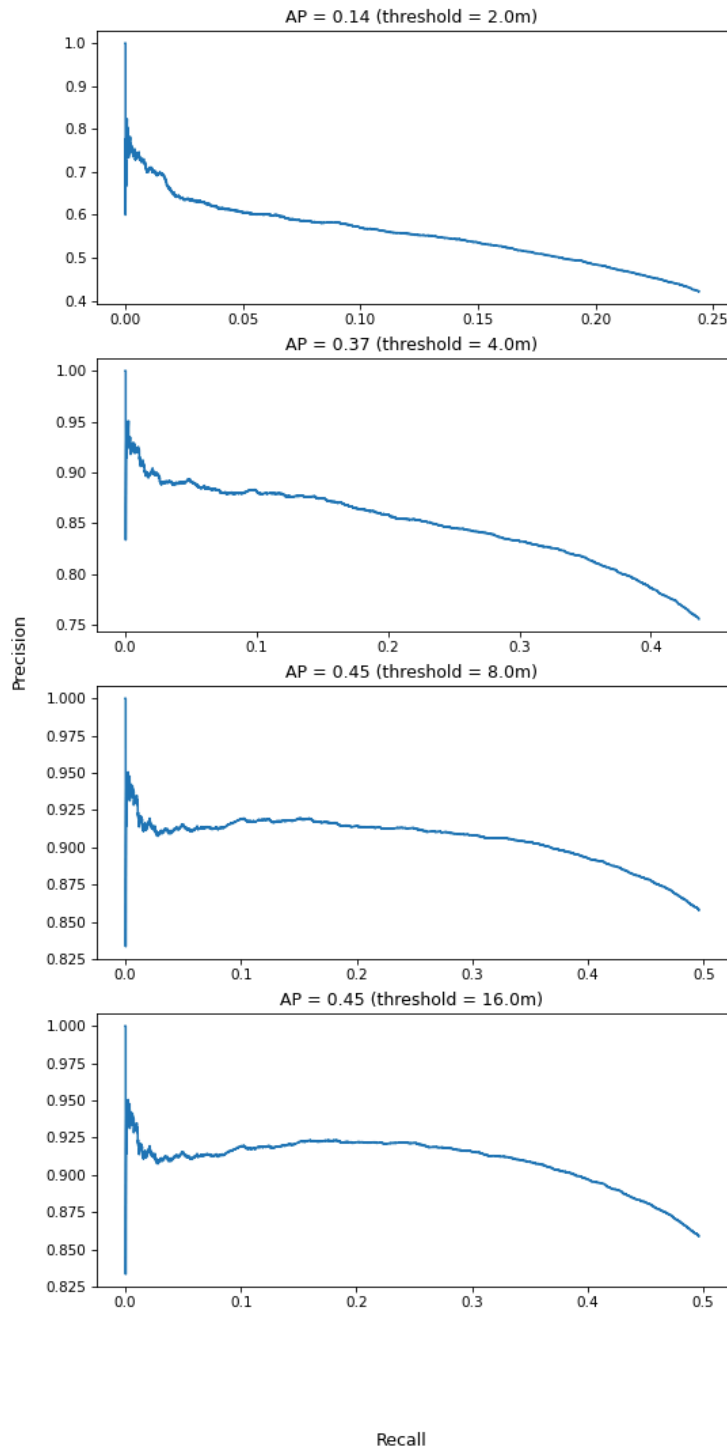


710.png:



Part 2.3.3 Plots + Discussions

Precision-Recall Curves



Discussion

As threshold increases from 2 to 16, the average precision increases. Moreover, the shape of the curve becomes flatter for a wider range of recalls and that precision only starts to drop at a bigger recall value.

Part 2.3.4

How is ground truth for the metrics generated?

Step 1: Raw data extraction

- Labels are being extracted from the raw Pandaset which contains either the training data or the validation data

Step 2: Projecting label data to voxelized frame

- We remove all the detections that is not within the voxelized frame (i.e: detections must be within (x_{min}, x_{max}) ; (y_{min}, y_{max}) and (z_{min}, z_{max}) which these numbers are set in the config)
- Then, we project the 3d label data (centroids) to become an image coordinate, with the formulas in 2.1.1 in the x, y direction (since image is only 2D) and adjust the box size based on the step size configured by dividing original label box size by step size
- This then gives us the labels where the model targets are from

Step 3: Getting Model Targets

- After the labels are produced, we will build a 7 X H X W tensor which contains the heatmap, , headings, bounding box sizes as columns
 - For each (centroid, sizes, yaw), we first used the heatmap to get coordinates with values that is larger than the heatmap_threshold and later compute the offsets, sizes, and sin, cos of yaw and only place those values in coordinates with heatmap value larger than the threshold in the 7 x H x W final target tensor

What does the above imply? Is there data that the model will never learn?

Since we only fill in values for coordinates with heatmap values that are larger than the threshold, there are going to be coordinates in the image where the target tensor is always going to be zero. What this means is that if there is a vehicle at those coordinates (probably at the sides; i.e: not the center of the image), since those coordinates would not generate a large enough heatmap value, the model could never detect vehicles at those coordinates. If we refer back to the testing visualizations, we can see that there are never red boxes near the top and bottom border of the images.

Part B

Motivation

What is the problem being Tackled

We are trying to tackle class imbalance, which occurs when a huge majority of data belongs to one class. With class imbalance, our original model, in which its loss function assumes data to be uniformly distributed in most cases, would become more biased towards the class which has more data. This implies that the model would perform worse in the class with less data. As a result, attempting to solve this problem, we decided to implement a more sophisticated loss function, the focal loss function which was proposed by Facebook AI Research in 2017, to penalize the model at the right time. [1]

Apart from that, we would also like to generalize the gaussian function that we were using. Initially, we were using the isotropic gaussian function to generate circular heatmaps for the detections. We were using the same scale for both the x and y direction. In other words, the sizes and the headings of the vehicles cannot be reflected in the heatmaps at all. Therefore, we implemented an anisotropic gaussian function to generate heatmaps that depend on more information (box sizes, and headings) of the detections.

Why is this problem relevant

This problem is relevant to our assignment as our major task is to detect cars. This means there are going to be two labels: a vehicle is centered at this pixel and a vehicle is not centered at this pixel. Looking at the detections.png that were generated, we know that there are only a few pixels that have a vehicle centered around it. Most of the pixels do not have detections at it, and therefore there is class imbalance in our dataset with regards to our detection task. Moreover, implementing anisotropic gaussian is also relevant to our task, because the size of x and size of y of the bounding box are different. Further, for some of the vehicles, they were also moving towards a significantly big angle (example: the car detected at [298, 220]). Therefore, using an anisotropic gaussian can create a heatmap that represents the data better.

Briefly explain other approaches to this problem

Apart from the focal loss function, there are also a few other ways to handle the problem of class imbalance. The first one is to apply random undersampling and oversampling across data from each of the classes. [2] Random under-sampling refers to the method of random sampling within the majority class while random over-sampling refers to random duplication of the minority class. Therefore, by reducing the difference of the amount of data in the classes, the imbalance would be decreased and hence improve the accuracy.

On the other hand, there is also another approach called data augmentation to solve this problem, which involves increasing the amount of data artificially by using the original data. One way to generate data is to use Generative adversarial networks. For example, Sallab et al have attempted to produce realistic LiDAR data from simulations. [3] Similar to the undersampling and oversampling method, it attempts to solve the problem of class imbalance through increasing the number of data points in the minority classes.

Techniques

Describe the intuition for why your approach would help

We think that implementing focal loss could help reduce the bias caused by class imbalance. In the paper by Lin et al [cite], there is a 13.2%, 9.14% and 10.3% increase in the average precision for 3 different thresholds when we compare focal loss against the baseline online hard example mining loss function. Given the statistics, we believe that it is reasonable to do experiments on it and see whether it could improve our average precision.

In addition to this, we also believe that implementing an anisotropic gaussian function to generate the heatmap would improve the accuracy. This is because the original isotropic gaussian would blur the image features that were captured in the data such as the headings and the bounding box sizes, while the anisotropic gaussian function captures this information. Therefore, we think that it might be better at capturing the location of the bounding boxes, and the exact angle of the vehicles.

Articulate your approach mathematically/algorithmically

Focal loss

Step 0: Define the predictions (which already has a max value of 1; which prediction indicates the probability of a vehicle at that pixel) to be the probability p

Step 1: Define p_t

$$p_t = p \text{ if } target == 1; p_t = 1 - p \text{ otherwise};$$

Step 2: Define α_t in a similar manner as p_t . In which $\alpha = 0.25$ as suggested in the paper.

$$\alpha_t = \alpha \text{ if } target == 1; \alpha_t = 1 - \alpha \text{ otherwise};$$

Step 3: Afterwards we compute the focal loss as described by Lin et al [cite], where $\gamma = 2$ as suggested.

$$FL(p_t) = \alpha_t * (1 - p_t)^\gamma * (-\log(p_t))$$

Step 4: Compute the mean of the focal loss of all pixels in the heatmap

```
def heatmap_focal_loss(targets: Tensor, predictions: Tensor, gamma: float, alpha: float):
    # p_t = p when target = 1 and 1-p when target = 0
    p = predictions
    p_t = p * targets + (1-p) * (1- targets)

    alpha_t = alpha * targets + (1-alpha) * (1 - targets)
    focal_loss = alpha_t * (1-p_t)**(gamma) * -(torch.log(p_t))

    return focal_loss.mean()
```

Anisotropic Gaussian

Step 0: Define the a, b, c in the following manner:

$$a = \frac{\cos^2(\theta)}{\text{scale}_x} + \frac{\sin^2(\theta)}{\text{scale}_y}$$
$$b = \frac{2 \cos(\theta) \sin(\theta)}{2\text{scale}_y} - \frac{2 \sin(\theta) \cos(\theta)}{2\text{scale}_x}$$
$$c = \frac{\cos^2(\theta)}{\text{scale}_y} + \frac{\sin^2(\theta)}{\text{scale}_x}$$

Step 1: Determine gaussian equation to be:

$$e^{-(a(x-x_0)^2 + 2b(x-x_0)(y-y_0) + c(y-y_0)^2)}$$

Where x_0, y_0 represents the x coordinate and the y coordinate of the centroids, and scale_x & scale_y are determined by the bounding box size and headings is a vector that contains $\sin(\text{yaw})$ and $\cos(\text{yaw})$.

```
def create_general_heatmap(grid_coords: Tensor, center: Tensor, scale_x: float, scale_y: float, headings: Tensor):
    x, y = grid_coords[:, :, 0], grid_coords[:, :, 1]
    sin, cos = headings[0], headings[1]
    a = cos**2/scale_x + sin**2/scale_y
    b = -((2*sin*cos)/(2*scale_x)) + (2*sin*cos)/(2*scale_y)
    c = sin**2/scale_x + cos**2/scale_y
    gaussianed = torch.exp(torch.neg(a*(x-center[0])**2 + c*(y-center[1])**2 + 2*b*(x-center[0])*(y-center[1])))

    return gaussianed
```

```
scale_x = 2*(x_size**2) / self._heatmap_norm_scale
scale_y = 2*(y_size**2) / self._heatmap_norm_scale
```

Evaluation

Articulate your evaluation plan mathematically/algorithmically

Experiment 1: Determining the effectiveness of sole focal loss

Step 1: Train model with different alpha, gamma pairs = [(0.25, 2), (0.25, 1), (0.25, 3), (0.1, 3), (0.2, 3)]

Step 2: Evaluate the trained model using Part A code and generate PR curve

Step 3: Compare the best AP and compute the percentage change with the original best AP generated by Part A code with the following formula

$$\frac{AP_{PartB} - AP_{PartA}}{AP_{PartA}}$$

Experiment 2: Determining the effectiveness of sole anisotropic gaussian

Step 1: Train model with different scale_x , scale_y pairs = [($x_size^2/\text{heatmap_norm_scale}$, $y_size^2/\text{heatmap_norm_scale}$), ($2*x_size^2/\text{heatmap_norm_scale}$, $2*y_size^2/\text{heatmap_norm_scale}$)]

Step 2: Evaluate the trained model using Part A code and generate PR curve

Step 3: Compare the best AP and compute the percentage change with the original best AP generated by Part A code with the following formula

$$\frac{AP_{PartB} - AP_{PartA}}{AP_{PartA}}$$

Experiment 3: Determining the effectiveness of focal loss + anisotropic gaussian

Step 1: Select the best set of hyperparameters from the above 2 experiments and adopt them for training

Step 2: Evaluate the trained model using Part A code and generate PR curve

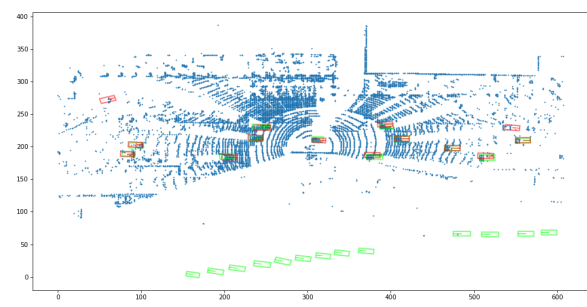
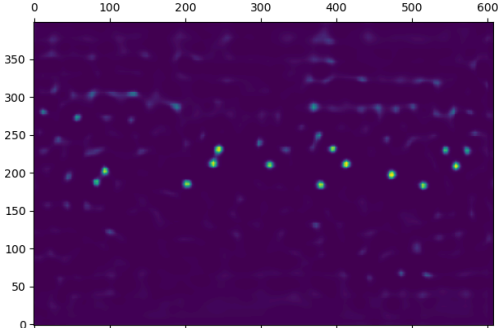
Step 3: Compare the best AP and compute the percentage change with the original best AP generated by Part A code with the following formula

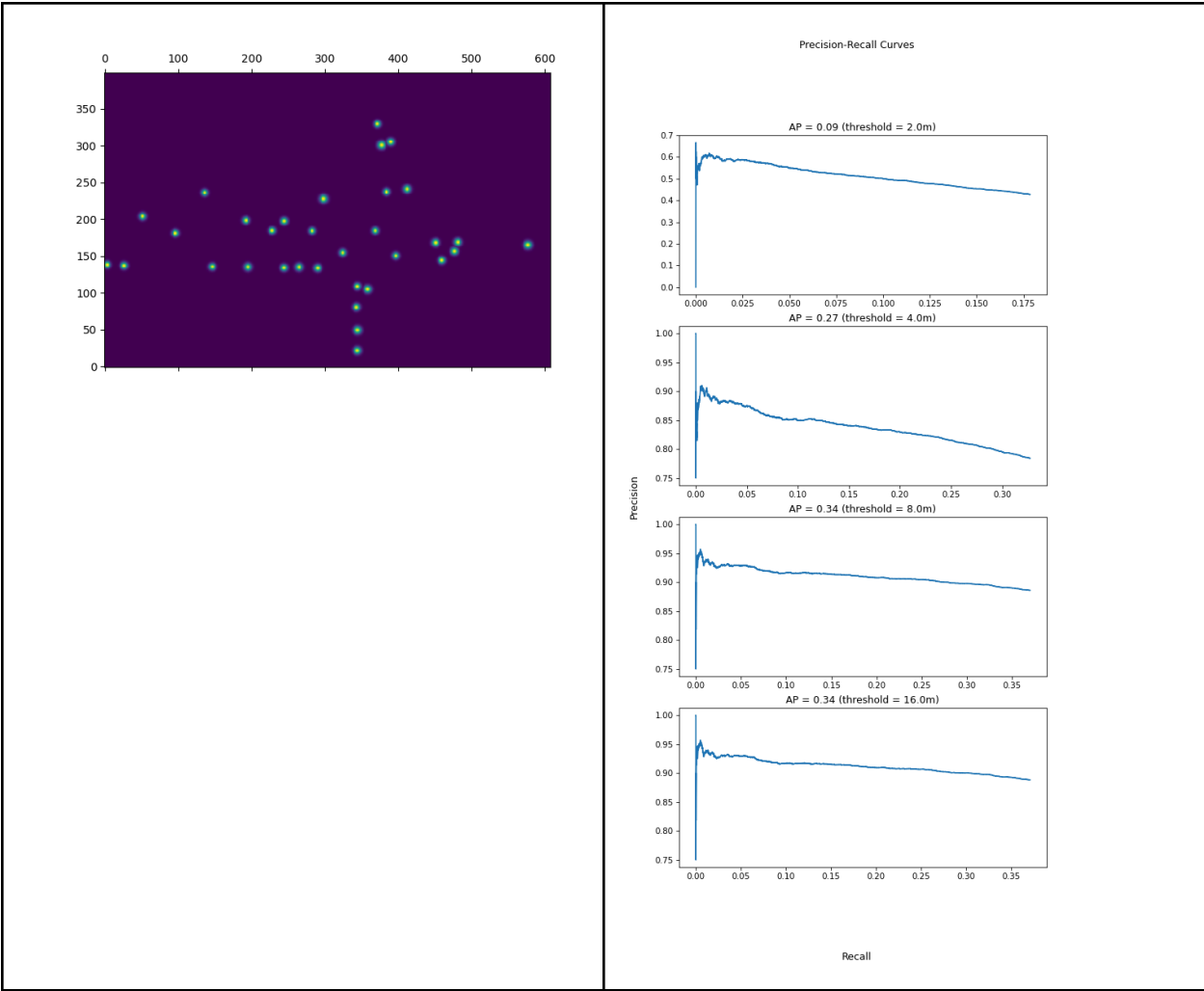
$$\frac{AP_{PartB} - AP_{PartA}}{AP_{PartA}}$$

Show compelling qualitative results/comparisons

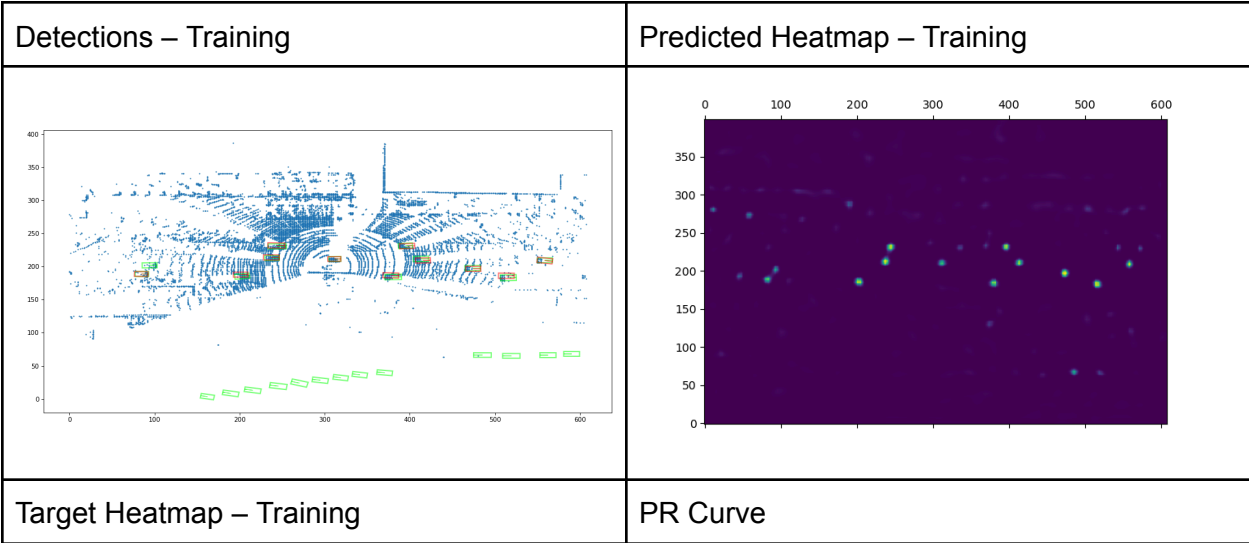
Experiment 1: Visualizations Generated From model using just focal loss

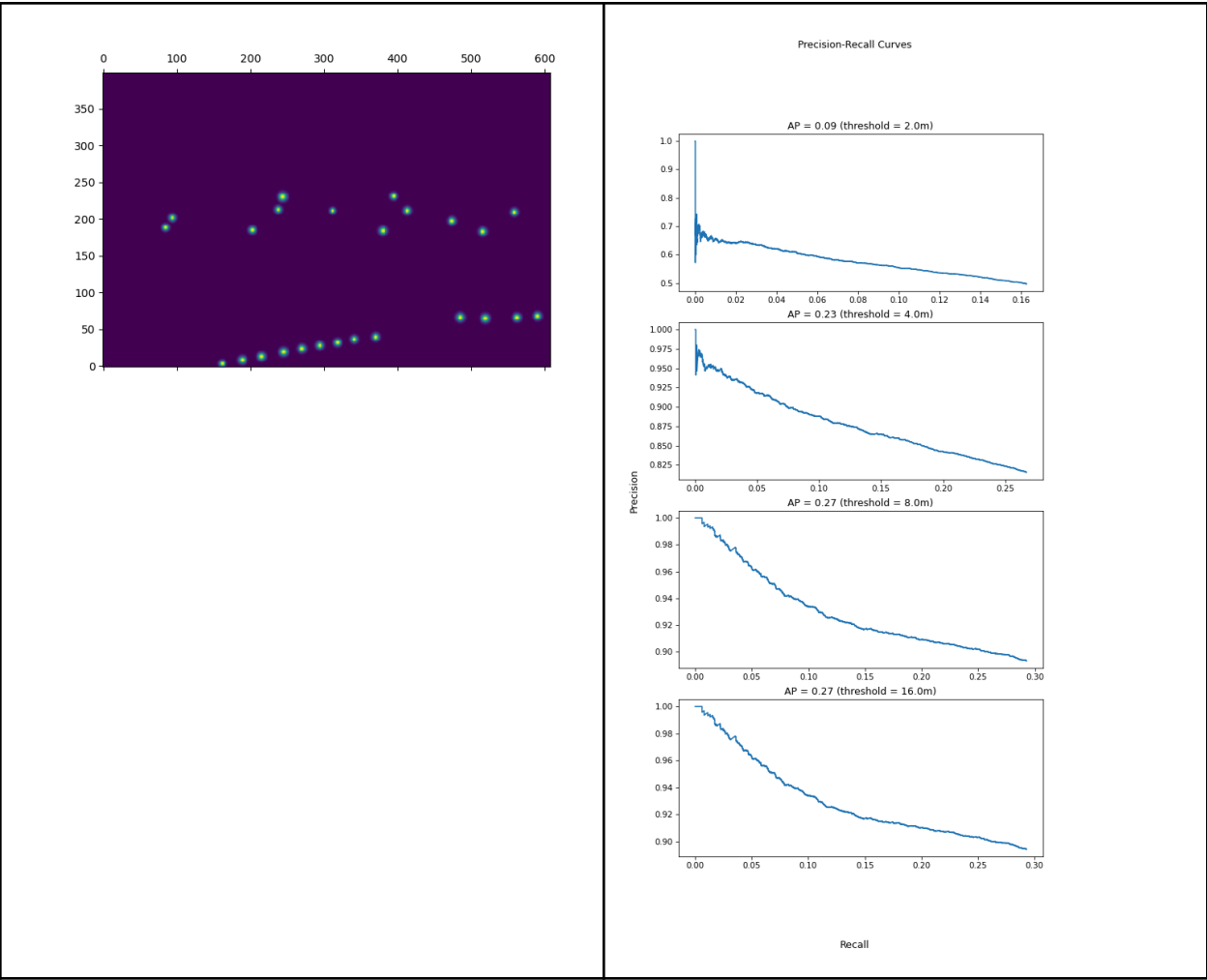
Gamma = 2, alpha = 0.25

Detections – Training	Predicted Heatmap – Training
	
Target Heatmap – Training	PR Curve

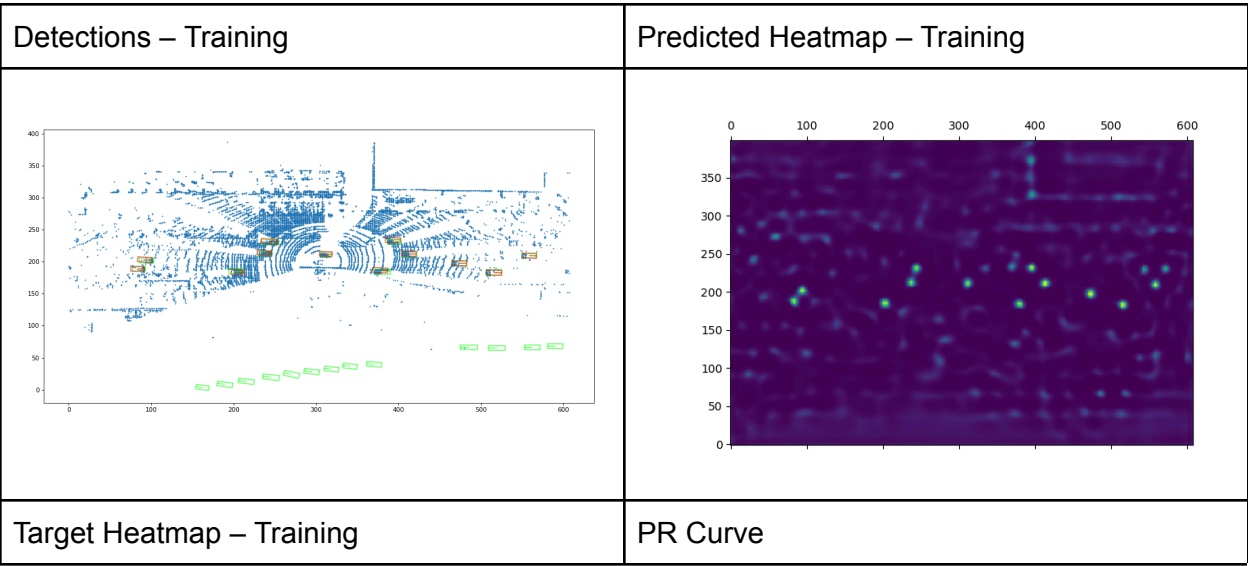


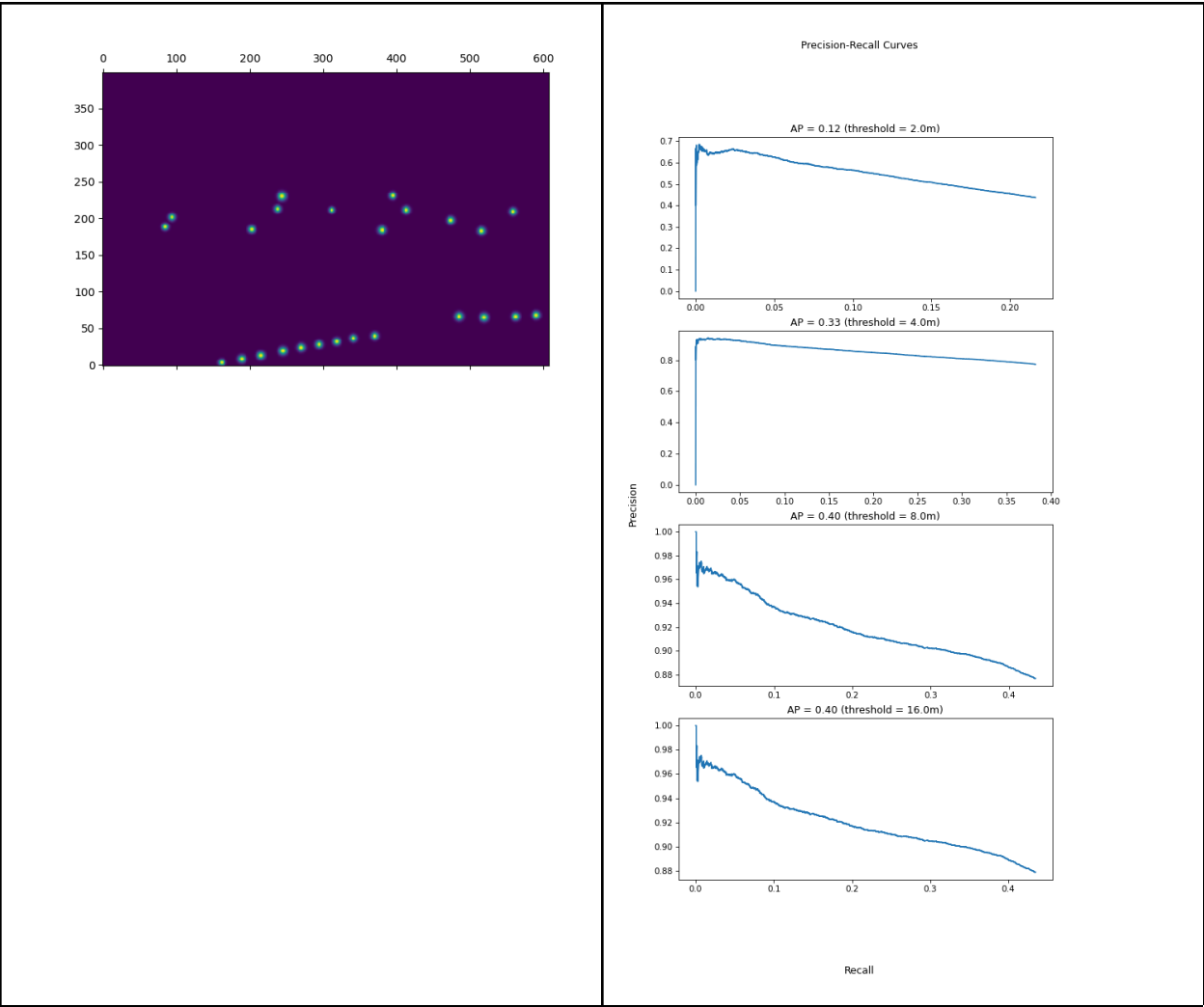
Gamma = 1, alpha = 0.25



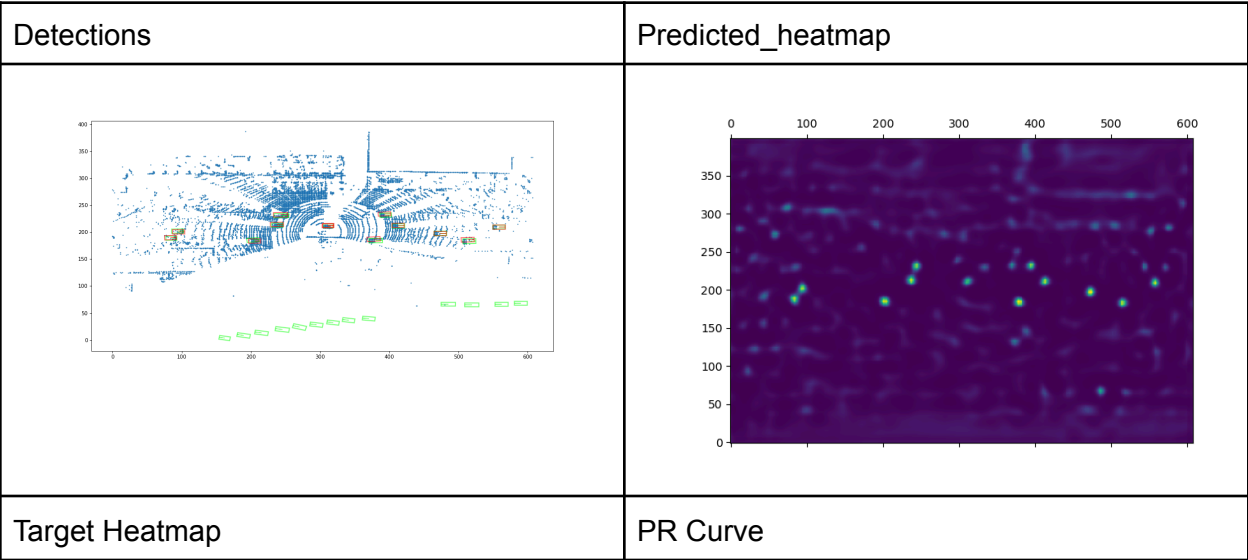


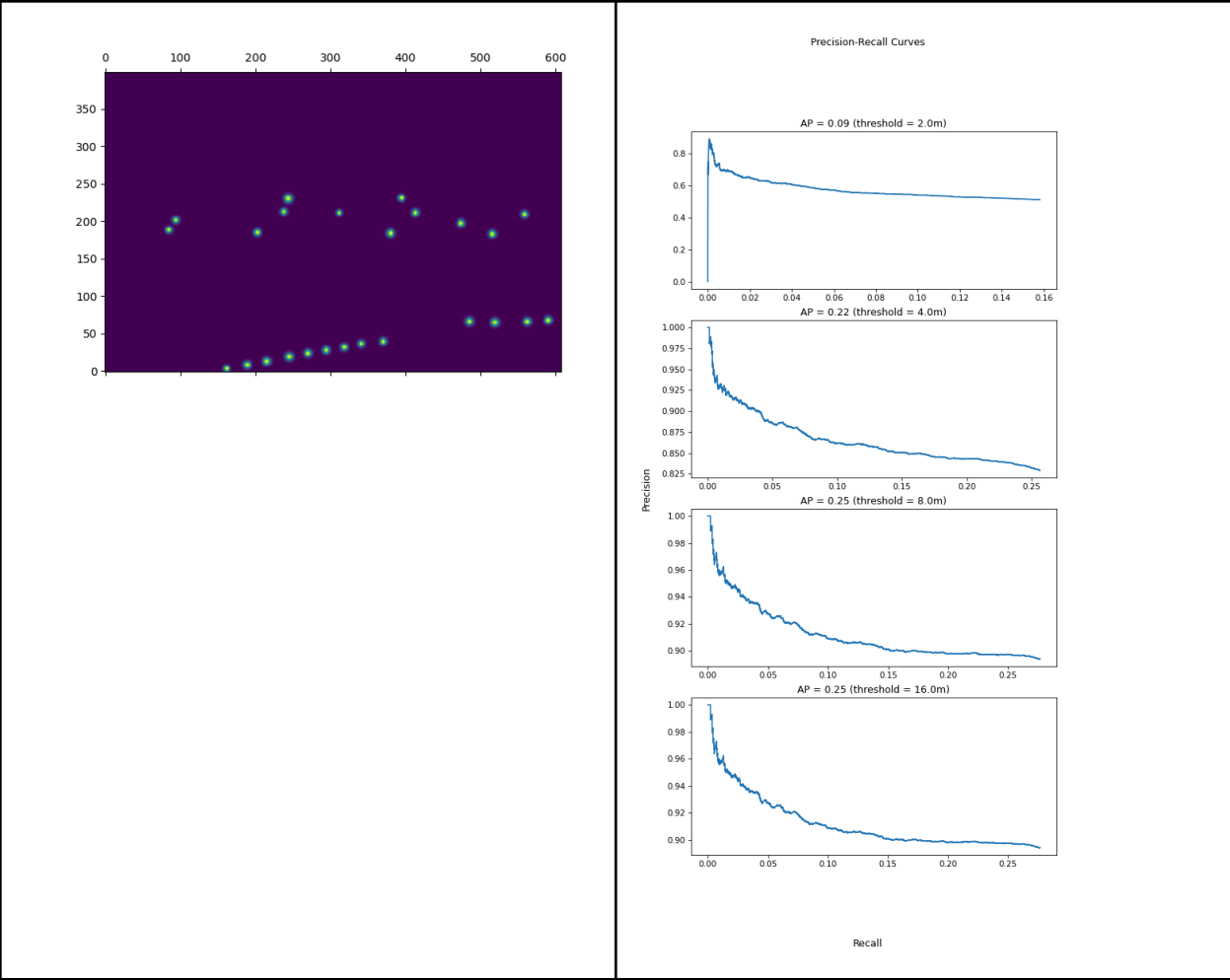
Gamma = 3, alpha = 0.25



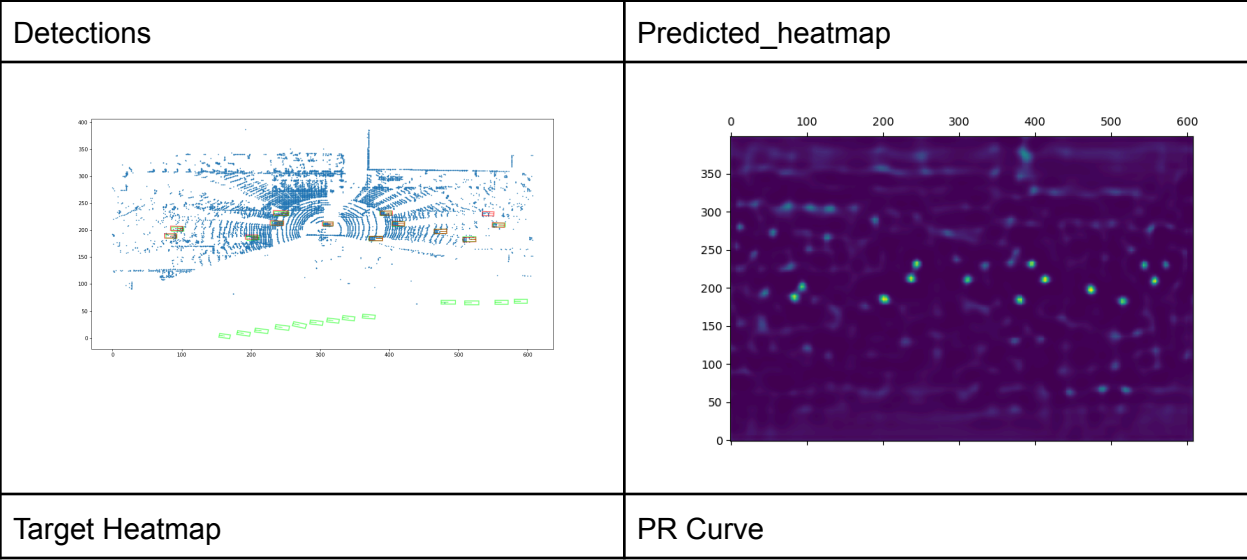


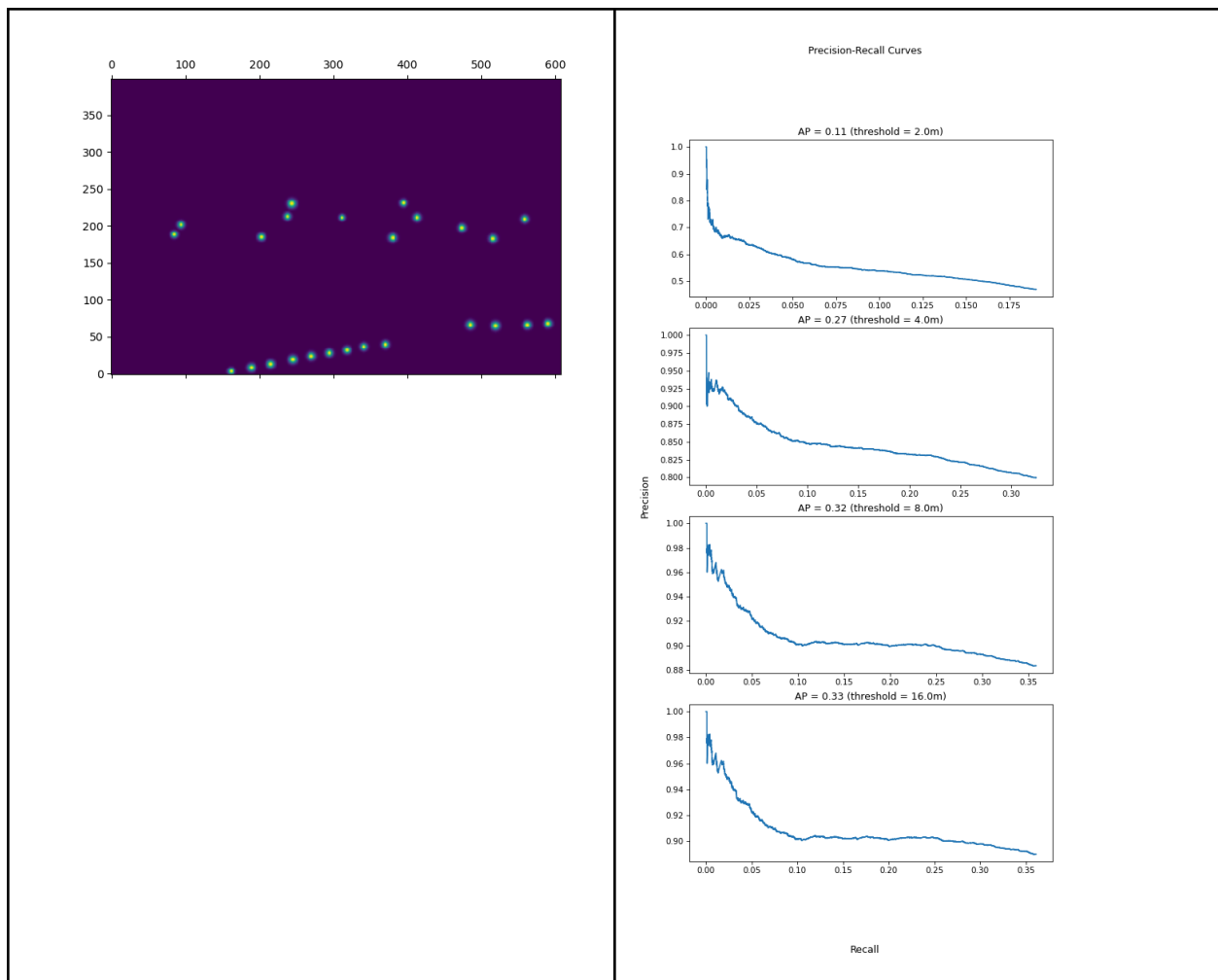
Gamma = 3, alpha = 0.1





Gamma = 3, alpha = 0.2





Discussion:

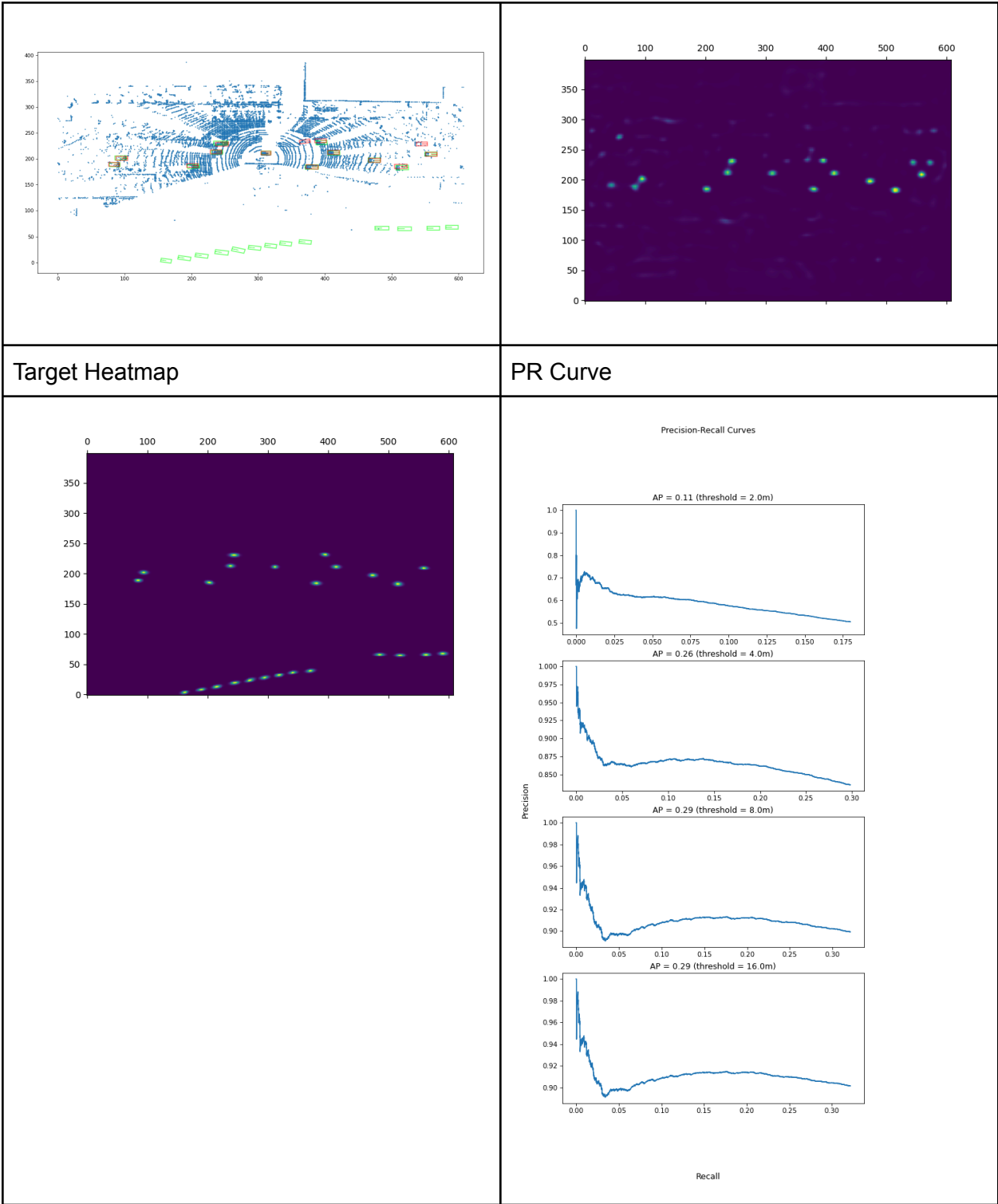
As shown from the above predicted_heatmaps and detections, more vehicles are detected. When gamma was increased from 1 to 2, not only were there more detections, there were also a few false positives from the detections.png when gamma = 2. However, when we further increase the gamma from 2 to 3, the false positives were gone, and most detection that was in the middle of the image are validly detected. The PR curve aligns with the above observation, as the average precision value also increases as the gamma value increases.

According to the paper, a larger gamma would be better paired with smaller alpha. But the average precision decreased when we tried using lower alphas of 0.2 and 0.1, instead of 0.25.

Experiment 2: Visualizations Generated From model using just Anisotropic Gaussian

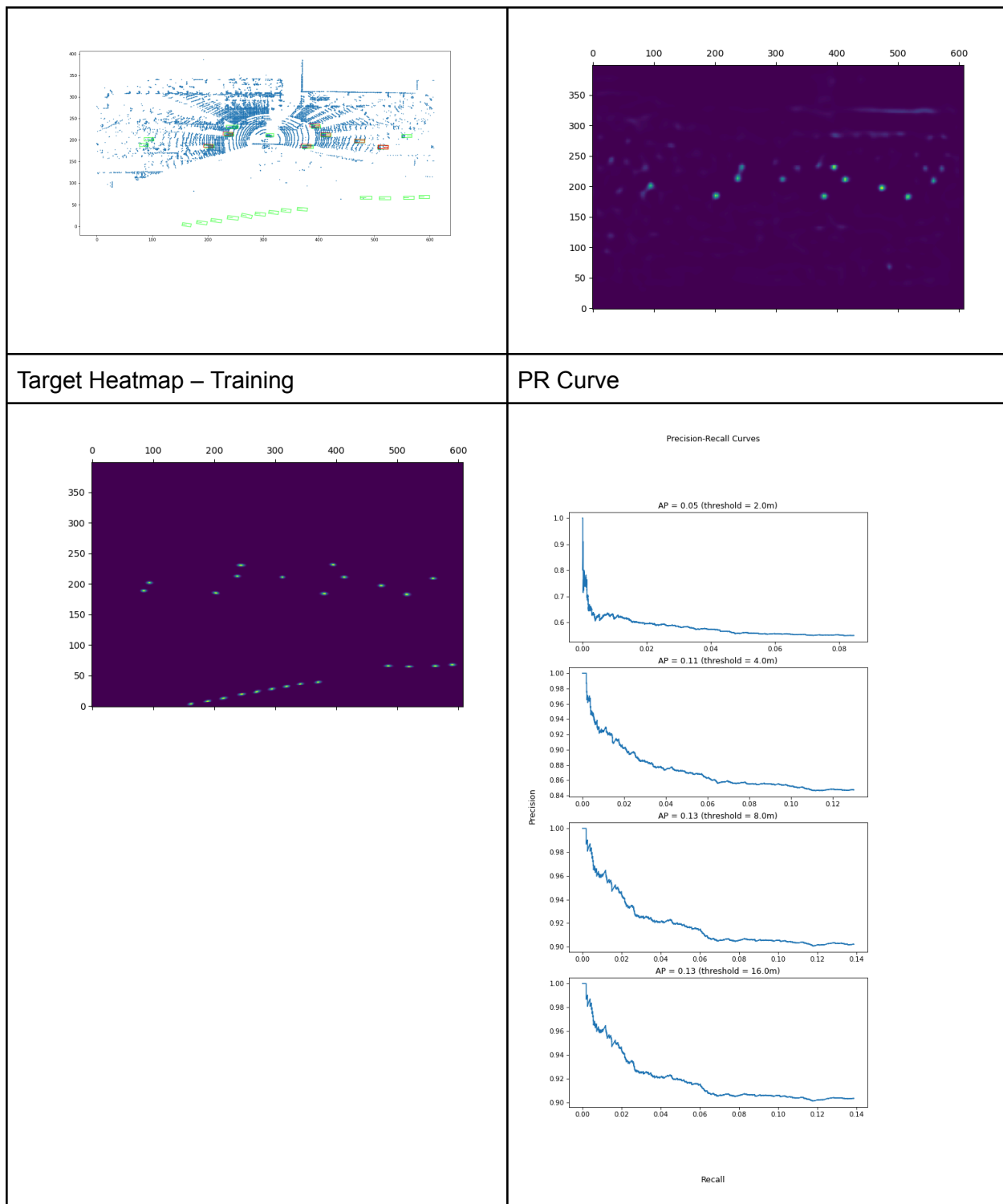
Scale x = $2 * (\text{boxsize_x}^2) / \text{heatmap_norm_scale}$; Scale y = $2 * (\text{boxsize_y}^2) / \text{heatmap_norm_scale}$;

Detections	Predicted_heatmap
------------	-------------------



Scale x = $1 * (\text{boxsize_x}^2) / \text{heatmap_norm_scale}$; Scale y = $1 * (\text{boxsize_y}^2) / \text{heatmap_norm_scale}$

Detections – Training	Predicted Heatmap – Training
-----------------------	------------------------------



Discussions

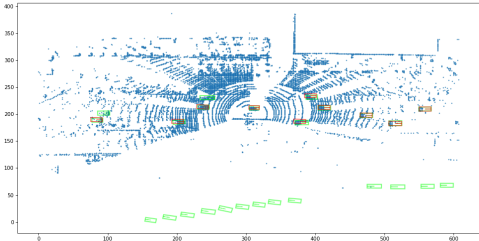
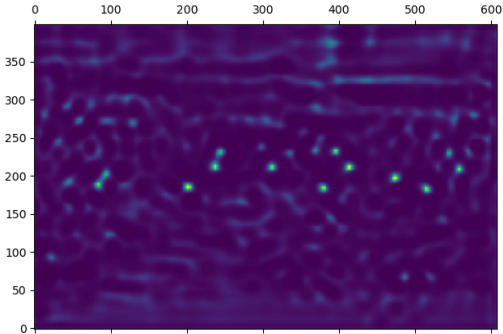
Using smaller $scale_x$ and $scale_y$ for the rotated anisotropic gaussian in creating heatmap gives a lower average precision in comparison to using the standard $scale_x$ and $scale_y$ that we've been using for all other experiments.

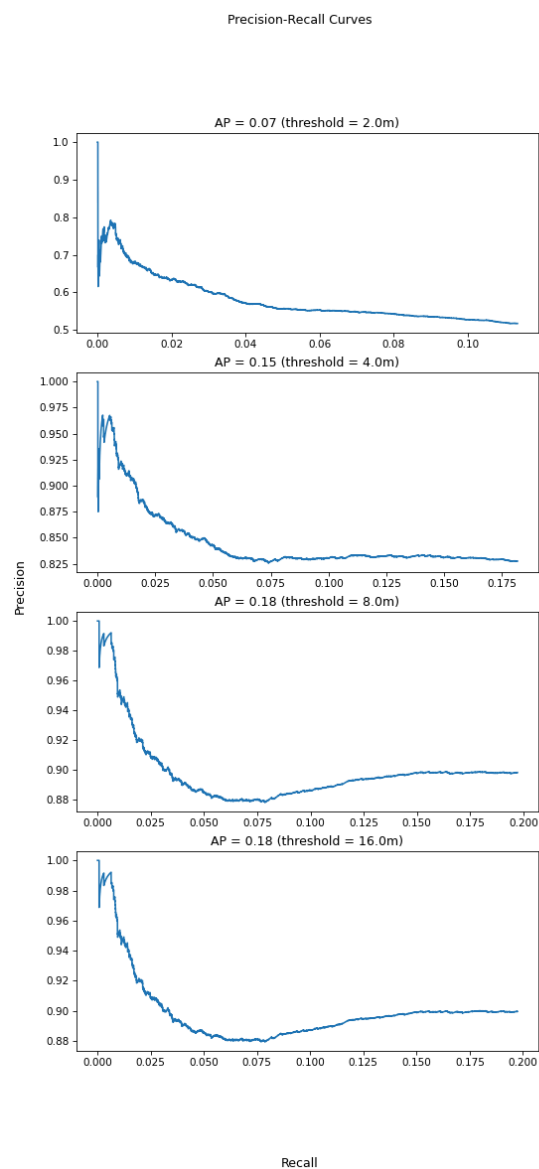
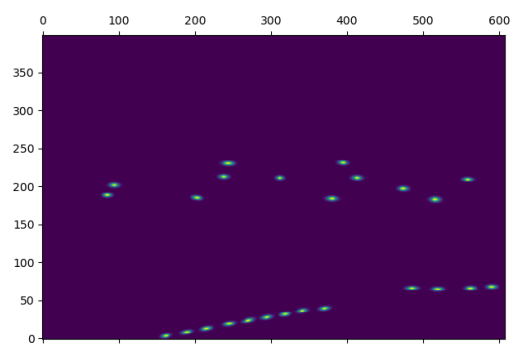
Experiment 3 Combining both anisotropic gaussian and focal loss

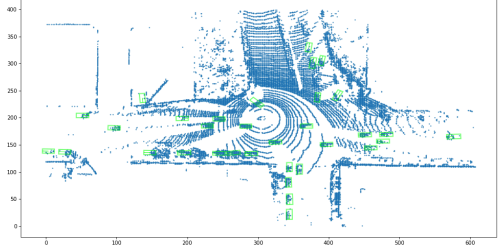
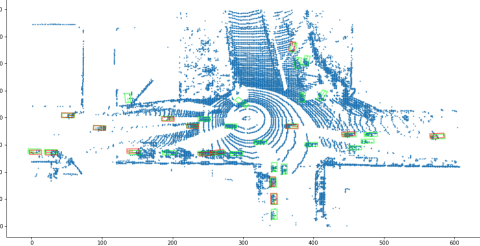
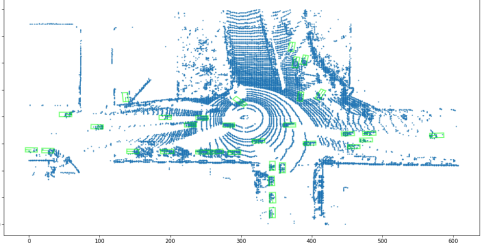
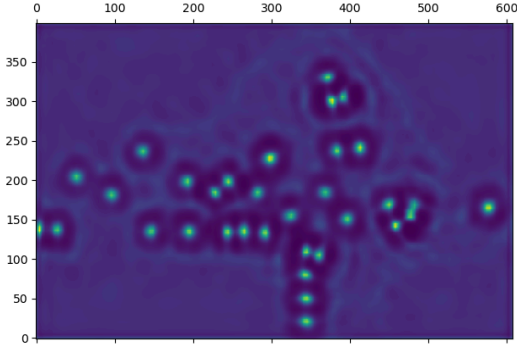
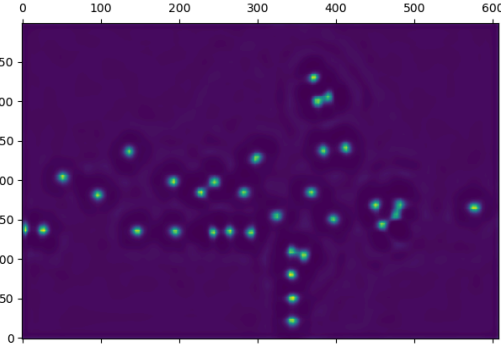
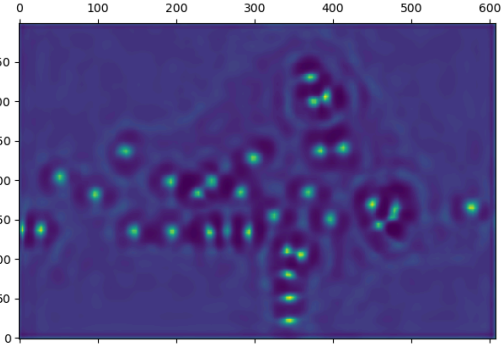
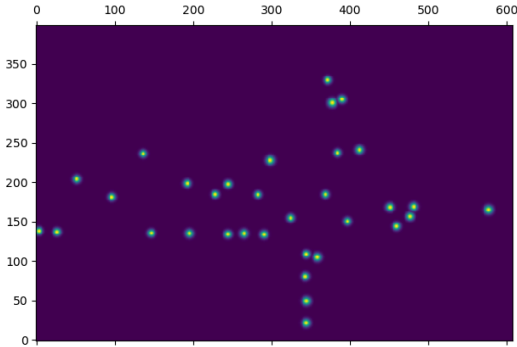
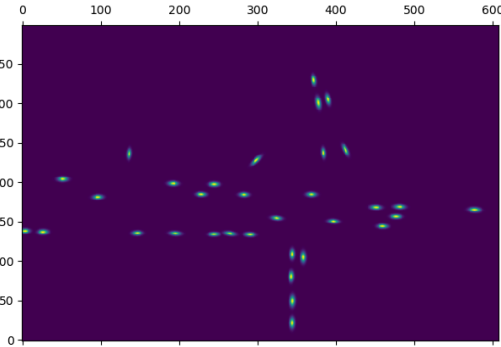
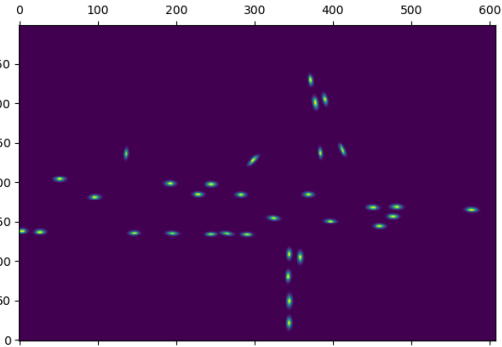
Settings:

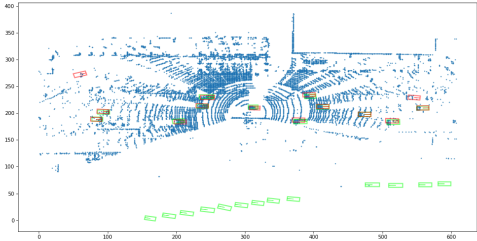
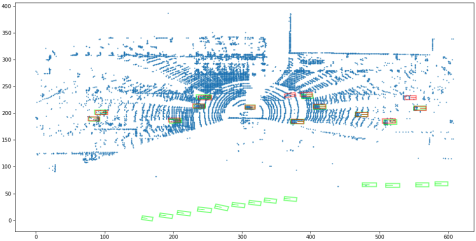
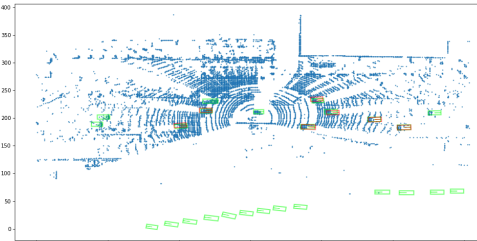
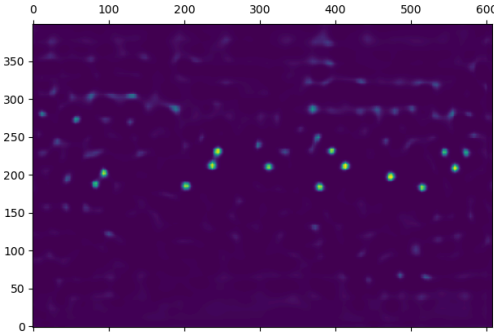
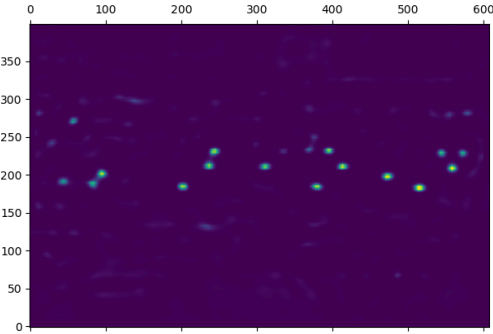
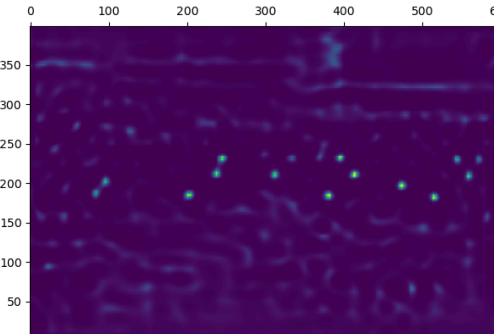
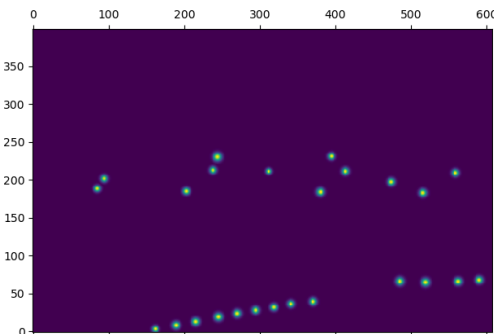
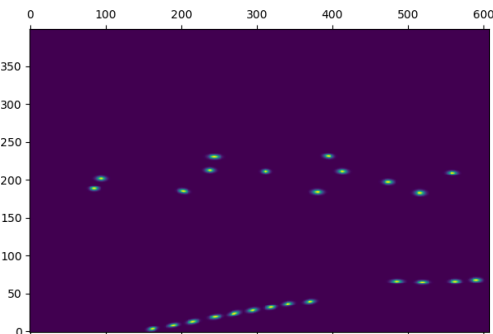
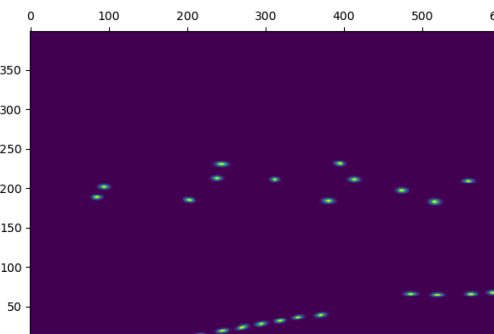
Gamma = 3

Alpha = 0.25
Scale_x = 2*(boxsize_x)^2 / heatmap_norm_scale
Scale_y = 2*(boxsize_y)^2/heatmap_norm_scale

Detections	Predicted_heatmap
	
Target Heatmap	PR Curve



Overfit	Focal loss	Rotated Anisotropic Gaussian	Focal loss & rotated Anisotropic Gaussian
detection.png			
predicted_heatmap			
target_heatmap			
	overfit detection.png for any experiment involving focal loss didn't output red boxes		

Train	Focal loss	Rotated Anisotropic Gaussian	Focal loss & rotated Anisotropic Gaussian
	(0.25, 2)		
detecti on.png			
predict ed_hea tmap			
target_ heatma p			
Loss	1080it [06:46, 2.66it/s] Epoch 0 [99/1080]: Loss - 66.0486 Heatmap Loss - 0.0042 Offset Loss - 4.1026 Size Loss - 17.2704 Heading Loss -	1080it [07:15, 2.48it/s] Epoch 0 [99/1080]: Loss - 65.7276 Heatmap Loss - 0.0074 Offset Loss - 4.0131 Size Loss - 17.5644 Heading Loss -	1080it [07:07, 2.53it/s] Epoch 0 [99/1080]: Loss - 64.9516 Heatmap Loss - 0.0041 Offset Loss - 4.0190 Size Loss - 17.8061 Heading Loss -

0.0734 Epoch 0 [199/1080]: Loss - 64.1971 Heatmap Loss - 0.0030 Offset Loss - 3.7779 Size Loss - 4.2432 Heading Loss - 0.2187 Epoch 0 [299/1080]: Loss - 56.6805 Heatmap Loss - 0.0031 Offset Loss - 3.7537 Size Loss - 2.6681 Heading Loss - 0.1616 Epoch 0 [399/1080]: Loss - 41.2392 Heatmap Loss - 0.0035 Offset Loss - 2.9159 Size Loss - 1.2109 Heading Loss - 0.1052 Epoch 0 [499/1080]: Loss - 38.8722 Heatmap Loss - 0.0024 Offset Loss - 2.7760 Size Loss - 1.2614 Heading Loss - 0.0961 Epoch 0 [599/1080]: Loss - 33.0175 Heatmap Loss - 0.0024 Offset Loss - 2.6636 Size Loss - 0.7433 Heading Loss - 0.0540 Epoch 0 [699/1080]: Loss - 30.8878 Heatmap Loss - 0.0023 Offset Loss - 2.1383 Size Loss - 0.8989 Heading Loss - 0.0837 Epoch 0 [799/1080]: Loss - 21.0422 Heatmap Loss - 0.0021 Offset Loss - 1.7466 Size Loss - 0.7229 Heading Loss - 0.0264 Epoch 0 [899/1080]: Loss - 22.6813 Heatmap Loss - 0.0024 Offset Loss - 1.6762 Size Loss - 0.6023 Heading Loss - 0.0508 Epoch 0 [999/1080]: Loss - 16.4169 Heatmap Loss - 0.0013 Offset Loss - 1.3693 Size Loss - 0.3244 Heading Loss - 0.0227 1080it [06:44, 2.67it/s] Epoch 1 [99/1080]: Loss - 23.0779 Heatmap Loss - 0.0016 Offset Loss - 1.7221 Size Loss - 0.8014 Heading Loss - 0.0490 Epoch 1 [199/1080]: Loss - 11.9609 Heatmap Loss - 0.0022 Offset Loss - 1.0500 Size Loss - 0.5672 Heading Loss - 0.0068 Epoch 1 [299/1080]: Loss - 18.2398 Heatmap Loss - 0.0014 Offset Loss - 1.4534 Size Loss - 0.6253 Heading Loss - 0.0294 Epoch 1 [399/1080]: Loss - 23.5115 Heatmap Loss - 0.0031 Offset Loss - 2.0057 Size Loss - 0.5860 Heading Loss - 0.0256 Epoch 1 [499/1080]: Loss - 19.8188 Heatmap Loss - 0.0016 Offset Loss - 1.6782 Size Loss - 1.2342 Heading Loss - 0.0164 Epoch 1 [599/1080]: Loss - 21.6117 Heatmap Loss - 0.0026 Offset Loss - 1.7532 Size Loss - 0.5047 Heading Loss - 0.0331 Epoch 1 [699/1080]: Loss - 21.3371 Heatmap Loss - 0.0020 Offset Loss - 1.8383 Size Loss - 0.6125 Heading Loss - 0.0214 Epoch 1 [799/1080]: Loss - 19.2364 Heatmap Loss - 0.0028 Offset Loss - 1.4290 Size Loss - 0.5628 Heading Loss - 0.0410 Epoch 1 [899/1080]: Loss - 12.9770 Heatmap Loss - 0.0013 Offset Loss - 1.1424 Size Loss - 0.5778 Heading Loss - 0.0084 Epoch 1 [999/1080]: Loss - 16.5905 Heatmap Loss -	0.0729 Epoch 0 [199/1080]: Loss - 60.9306 Heatmap Loss - 0.0050 Offset Loss - 3.4456 Size Loss - 3.8974 Heading Loss - 0.2208 Epoch 0 [299/1080]: Loss - 56.5458 Heatmap Loss - 0.0048 Offset Loss - 3.6929 Size Loss - 2.4174 Heading Loss - 0.1672 Epoch 0 [399/1080]: Loss - 42.2982 Heatmap Loss - 0.0056 Offset Loss - 3.0081 Size Loss - 1.2605 Heading Loss - 0.1040 Epoch 0 [499/1080]: Loss - 37.8330 Heatmap Loss - 0.0036 Offset Loss - 2.5371 Size Loss - 1.7203 Heading Loss - 0.1038 Epoch 0 [599/1080]: Loss - 36.2555 Heatmap Loss - 0.0036 Offset Loss - 2.9289 Size Loss - 0.8303 Heading Loss - 0.0578 Epoch 0 [699/1080]: Loss - 32.4321 Heatmap Loss - 0.0039 Offset Loss - 2.2894 Size Loss - 1.1985 Heading Loss - 0.0795 Epoch 0 [799/1080]: Loss - 23.9355 Heatmap Loss - 0.0038 Offset Loss - 2.0221 Size Loss - 0.6560 Heading Loss - 0.0268 Epoch 0 [899/1080]: Loss - 21.2129 Heatmap Loss - 0.0042 Offset Loss - 1.5830 Size Loss - 0.6011 Heading Loss - 0.0436 Epoch 0 [999/1080]: Loss - 15.9741 Heatmap Loss - 0.0024 Offset Loss - 1.2563 Size Loss - 0.3866 Heading Loss - 0.0278 1080it [07:06, 2.53it/s] Epoch 1 [99/1080]: Loss - 25.2449 Heatmap Loss - 0.0028 Offset Loss - 1.8756 Size Loss - 0.8222 Heading Loss - 0.0539 Epoch 1 [199/1080]: Loss - 12.9269 Heatmap Loss - 0.0041 Offset Loss - 1.0812 Size Loss - 0.6735 Heading Loss - 0.0103 Epoch 1 [299/1080]: Loss - 18.2315 Heatmap Loss - 0.0024 Offset Loss - 1.4050 Size Loss - 0.8677 Heading Loss - 0.0307 Epoch 1 [399/1080]: Loss - 23.8342 Heatmap Loss - 0.0052 Offset Loss - 2.0819 Size Loss - 0.5794 Heading Loss - 0.0192 Epoch 1 [499/1080]: Loss - 19.6377 Heatmap Loss - 0.0027 Offset Loss - 1.6629 Size Loss - 0.7757 Heading Loss - 0.0196 Epoch 1 [599/1080]: Loss - 19.0767 Heatmap Loss - 0.0042 Offset Loss - 1.6075 Size Loss - 0.3702 Heading Loss - 0.0222 Epoch 1 [699/1080]: Loss - 21.5403 Heatmap Loss - 0.0031 Offset Loss - 1.8363 Size Loss - 0.5573 Heading Loss - 0.0231 Epoch 1 [799/1080]: Loss - 20.9590 Heatmap Loss - 0.0042 Offset Loss - 1.6325 Size Loss - 0.4770 Heading Loss - 0.0374 Epoch 1 [899/1080]: Loss - 12.0545 Heatmap Loss - 0.0021 Offset Loss - 1.0813 Size Loss - 0.4842 Heading Loss - 0.0054 Epoch 1 [999/1080]: Loss - 17.3174 Heatmap Loss -	0.0655 Epoch 0 [199/1080]: Loss - 60.4388 Heatmap Loss - 0.0022 Offset Loss - 3.4645 Size Loss - 3.8098 Heading Loss - 0.2176 Epoch 0 [299/1080]: Loss - 57.7623 Heatmap Loss - 0.0023 Offset Loss - 3.7180 Size Loss - 2.5989 Heading Loss - 0.1776 Epoch 0 [399/1080]: Loss - 41.5853 Heatmap Loss - 0.0026 Offset Loss - 2.9693 Size Loss - 0.9345 Heading Loss - 0.1070 Epoch 0 [499/1080]: Loss - 36.0434 Heatmap Loss - 0.0019 Offset Loss - 2.4915 Size Loss - 1.2944 Heading Loss - 0.0964 Epoch 0 [599/1080]: Loss - 34.5090 Heatmap Loss - 0.0017 Offset Loss - 2.8443 Size Loss - 0.6338 Heading Loss - 0.0526 Epoch 0 [699/1080]: Loss - 32.2287 Heatmap Loss - 0.0018 Offset Loss - 2.2191 Size Loss - 1.3890 Heading Loss - 0.0847 Epoch 0 [799/1080]: Loss - 24.3106 Heatmap Loss - 0.0017 Offset Loss - 2.0314 Size Loss - 0.6238 Heading Loss - 0.0320 Epoch 0 [899/1080]: Loss - 21.6015 Heatmap Loss - 0.0019 Offset Loss - 1.6259 Size Loss - 0.4164 Heading Loss - 0.0474 Epoch 0 [999/1080]: Loss - 15.4141 Heatmap Loss - 0.0011 Offset Loss - 1.2300 Size Loss - 0.5243 Heading Loss - 0.0248 1080it [07:03, 2.55it/s] Epoch 1 [99/1080]: Loss - 24.4115 Heatmap Loss - 0.0012 Offset Loss - 1.8581 Size Loss - 0.8169 Heading Loss - 0.0489 Epoch 1 [199/1080]: Loss - 12.0204 Heatmap Loss - 0.0018 Offset Loss - 1.0678 Size Loss - 0.4511 Heading Loss - 0.0071 Epoch 1 [299/1080]: Loss - 18.3929 Heatmap Loss - 0.0011 Offset Loss - 1.3568 Size Loss - 0.6381 Heading Loss - 0.0408 Epoch 1 [399/1080]: Loss - 22.7564 Heatmap Loss - 0.0024 Offset Loss - 1.9779 Size Loss - 0.7082 Heading Loss - 0.0203 Epoch 1 [499/1080]: Loss - 19.6496 Heatmap Loss - 0.0013 Offset Loss - 1.6112 Size Loss - 0.6924 Heading Loss - 0.0272 Epoch 1 [599/1080]: Loss - 19.3021 Heatmap Loss - 0.0020 Offset Loss - 1.6378 Size Loss - 0.5048 Heading Loss - 0.0222 Epoch 1 [699/1080]: Loss - 21.3707 Heatmap Loss - 0.0016 Offset Loss - 1.8737 Size Loss - 0.5663 Heading Loss - 0.0191 Epoch 1 [799/1080]: Loss - 19.7470 Heatmap Loss - 0.0023 Offset Loss - 1.5945 Size Loss - 0.4722 Heading Loss - 0.0310 Epoch 1 [899/1080]: Loss - 13.3187 Heatmap Loss - 0.0011 Offset Loss - 1.1581 Size Loss - 0.5539 Heading Loss - 0.0108 Epoch 1 [999/1080]: Loss - 17.9690 Heatmap Loss - 0.0017
---	---	--

0.0022 Offset Loss - 1.4441 Size Loss - 0.5022 Heading Loss - 0.0143 1080it [06:40, 2.70it/s] Epoch 2 [99/1080]: Loss - 10.5200 Heatmap Loss - 0.0010 Offset Loss - 0.9794 Size Loss - 0.3497 Heading Loss - 0.0027 Epoch 2 [199/1080]: Loss - 15.6589 Heatmap Loss - 0.0015 Offset Loss - 1.2431 Size Loss - 0.6517 Heading Loss - 0.0243 Epoch 2 [299/1080]: Loss - 21.8514 Heatmap Loss - 0.0026 Offset Loss - 1.7604 Size Loss - 0.7686 Heading Loss - 0.0322 Epoch 2 [399/1080]: Loss - 11.2633 Heatmap Loss - 0.0015 Offset Loss - 0.9334 Size Loss - 0.4385 Heading Loss - 0.0134 Epoch 2 [499/1080]: Loss - 10.5874 Heatmap Loss - 0.0017 Offset Loss - 0.8933 Size Loss - 0.6151 Heading Loss - 0.0087 Epoch 2 [599/1080]: Loss - 12.0116 Heatmap Loss - 0.0027 Offset Loss - 1.0638 Size Loss - 0.3197 Heading Loss - 0.0078 Epoch 2 [699/1080]: Loss - 13.6193 Heatmap Loss - 0.0034 Offset Loss - 1.1921 Size Loss - 0.5188 Heading Loss - 0.0084 Epoch 2 [799/1080]: Loss - 12.0662 Heatmap Loss - 0.0015 Offset Loss - 1.0269 Size Loss - 0.4756 Heading Loss - 0.0117 Epoch 2 [899/1080]: Loss - 5.3719 Heatmap Loss - 0.0009 Offset Loss - 0.4532 Size Loss - 0.4016 Heading Loss - 0.0034 Epoch 2 [999/1080]: Loss - 5.5553 Heatmap Loss - 0.0012 Offset Loss - 0.4684 Size Loss - 0.3478 Heading Loss - 0.0040 1080it [06:42, 2.68it/s] Epoch 3 [99/1080]: Loss - 6.7591 Heatmap Loss - 0.0017 Offset Loss - 0.5806 Size Loss - 0.4654 Heading Loss - 0.0032 Epoch 3 [199/1080]: Loss - 7.9266 Heatmap Loss - 0.0019 Offset Loss - 0.6723 Size Loss - 0.2949 Heading Loss - 0.0072 Epoch 3 [299/1080]: Loss - 12.3965 Heatmap Loss - 0.0020 Offset Loss - 1.0047 Size Loss - 0.4164 Heading Loss - 0.0173 Epoch 3 [399/1080]: Loss - 11.3174 Heatmap Loss - 0.0019 Offset Loss - 0.9444 Size Loss - 0.4712 Heading Loss - 0.0121 Epoch 3 [499/1080]: Loss - 14.3281 Heatmap Loss - 0.0022 Offset Loss - 1.1878 Size Loss - 0.6155 Heading Loss - 0.0161 Epoch 3 [599/1080]: Loss - 11.9340 Heatmap Loss - 0.0011 Offset Loss - 1.0822 Size Loss - 0.3208 Heading Loss - 0.0069 Epoch 3 [699/1080]: Loss - 10.3829 Heatmap Loss - 0.0011 Offset Loss - 0.9396 Size Loss - 0.2461 Heading Loss - 0.0063 Epoch 3 [799/1080]: Loss - 14.3109 Heatmap Loss - 0.0025 Offset Loss - 1.2555 Size Loss - 0.4330 Heading	0.0033 Offset Loss - 1.4484 Size Loss - 0.4834 Heading Loss - 0.0202 1080it [07:03, 2.55it/s] Epoch 2 [99/1080]: Loss - 11.5608 Heatmap Loss - 0.0015 Offset Loss - 1.0684 Size Loss - 0.4122 Heading Loss - 0.0032 Epoch 2 [199/1080]: Loss - 14.1711 Heatmap Loss - 0.0023 Offset Loss - 1.0532 Size Loss - 0.5457 Heading Loss - 0.0286 Epoch 2 [299/1080]: Loss - 18.6271 Heatmap Loss - 0.0040 Offset Loss - 1.4264 Size Loss - 0.6569 Heading Loss - 0.0331 Epoch 2 [399/1080]: Loss - 12.8278 Heatmap Loss - 0.0020 Offset Loss - 1.0927 Size Loss - 0.6140 Heading Loss - 0.0108 Epoch 2 [499/1080]: Loss - 11.1639 Heatmap Loss - 0.0023 Offset Loss - 0.9323 Size Loss - 0.5714 Heading Loss - 0.0104 Epoch 2 [599/1080]: Loss - 11.4466 Heatmap Loss - 0.0039 Offset Loss - 0.9924 Size Loss - 0.3854 Heading Loss - 0.0075 Epoch 2 [699/1080]: Loss - 12.6193 Heatmap Loss - 0.0049 Offset Loss - 1.0984 Size Loss - 0.4204 Heading Loss - 0.0073 Epoch 2 [799/1080]: Loss - 10.9188 Heatmap Loss - 0.0019 Offset Loss - 0.8916 Size Loss - 0.4882 Heading Loss - 0.0132 Epoch 2 [899/1080]: Loss - 6.2741 Heatmap Loss - 0.0012 Offset Loss - 0.5406 Size Loss - 0.2820 Heading Loss - 0.0046 Epoch 2 [999/1080]: Loss - 7.9521 Heatmap Loss - 0.0016 Offset Loss - 0.6633 Size Loss - 0.6656 Heading Loss - 0.0049 1080it [07:05, 2.54it/s] Epoch 3 [99/1080]: Loss - 6.5679 Heatmap Loss - 0.0022 Offset Loss - 0.5789 Size Loss - 0.2754 Heading Loss - 0.0028 Epoch 3 [199/1080]: Loss - 7.8161 Heatmap Loss - 0.0026 Offset Loss - 0.6527 Size Loss - 0.3633 Heading Loss - 0.0067 Epoch 3 [299/1080]: Loss - 13.7845 Heatmap Loss - 0.0027 Offset Loss - 1.1455 Size Loss - 0.4213 Heading Loss - 0.0163 Epoch 3 [399/1080]: Loss - 11.3777 Heatmap Loss - 0.0027 Offset Loss - 0.9268 Size Loss - 0.4366 Heading Loss - 0.0141 Epoch 3 [499/1080]: Loss - 14.0272 Heatmap Loss - 0.0030 Offset Loss - 1.2038 Size Loss - 0.5437 Heading Loss - 0.0115 Epoch 3 [599/1080]: Loss - 10.0944 Heatmap Loss - 0.0015 Offset Loss - 0.8604 Size Loss - 0.4107 Heading Loss - 0.0092 Epoch 3 [699/1080]: Loss - 9.9476 Heatmap Loss - 0.0014 Offset Loss - 0.8656 Size Loss - 0.4300 Heading Loss - 0.0072 Epoch 3 [799/1080]: Loss - 15.6393 Heatmap Loss - 0.0034 Offset Loss - 1.3384 Size Loss - 0.4873 Heading	Offset Loss - 1.4130 Size Loss - 0.4234 Heading Loss - 0.0325 1080it [07:06, 2.53it/s] Epoch 2 [99/1080]: Loss - 12.3659 Heatmap Loss - 0.0008 Offset Loss - 1.1166 Size Loss - 0.8660 Heading Loss - 0.0026 Epoch 2 [199/1080]: Loss - 13.6137 Heatmap Loss - 0.0012 Offset Loss - 1.0545 Size Loss - 1.1382 Heading Loss - 0.0181 Epoch 2 [299/1080]: Loss - 17.3585 Heatmap Loss - 0.0020 Offset Loss - 1.3873 Size Loss - 0.8272 Heading Loss - 0.0246 Epoch 2 [399/1080]: Loss - 11.9652 Heatmap Loss - 0.0012 Offset Loss - 1.0248 Size Loss - 0.4087 Heading Loss - 0.0119 Epoch 2 [499/1080]: Loss - 10.3197 Heatmap Loss - 0.0013 Offset Loss - 0.9150 Size Loss - 0.3923 Heading Loss - 0.0065 Epoch 2 [599/1080]: Loss - 11.6023 Heatmap Loss - 0.0022 Offset Loss - 1.0185 Size Loss - 0.4054 Heading Loss - 0.0079 Epoch 2 [699/1080]: Loss - 12.6029 Heatmap Loss - 0.0026 Offset Loss - 1.0973 Size Loss - 0.5033 Heading Loss - 0.0087 Epoch 2 [799/1080]: Loss - 11.1158 Heatmap Loss - 0.0012 Offset Loss - 0.9509 Size Loss - 0.4557 Heading Loss - 0.0103 Epoch 2 [899/1080]: Loss - 4.9788 Heatmap Loss - 0.0008 Offset Loss - 0.4323 Size Loss - 0.2708 Heading Loss - 0.0031 Epoch 2 [999/1080]: Loss - 6.9343 Heatmap Loss - 0.0010 Offset Loss - 0.5857 Size Loss - 0.5322 Heading Loss - 0.0045 1080it [07:06, 2.53it/s] Epoch 3 [99/1080]: Loss - 6.4247 Heatmap Loss - 0.0015 Offset Loss - 0.5694 Size Loss - 0.2413 Heading Loss - 0.0034 Epoch 3 [199/1080]: Loss - 7.5144 Heatmap Loss - 0.0017 Offset Loss - 0.6448 Size Loss - 0.3100 Heading Loss - 0.0059 Epoch 3 [299/1080]: Loss - 12.8867 Heatmap Loss - 0.0017 Offset Loss - 1.0676 Size Loss - 0.4646 Heading Loss - 0.0158 Epoch 3 [399/1080]: Loss - 10.6737 Heatmap Loss - 0.0016 Offset Loss - 0.8976 Size Loss - 0.3747 Heading Loss - 0.0116 Epoch 3 [499/1080]: Loss - 13.1496 Heatmap Loss - 0.0018 Offset Loss - 1.1317 Size Loss - 0.5298 Heading Loss - 0.0113 Epoch 3 [599/1080]: Loss - 10.1970 Heatmap Loss - 0.0009 Offset Loss - 0.8978 Size Loss - 0.4039 Heading Loss - 0.0073 Epoch 3 [699/1080]: Loss - 9.8752 Heatmap Loss - 0.0009 Offset Loss - 0.9142 Size Loss - 0.2417 Heading Loss - 0.0040 Epoch 3 [799/1080]: Loss - 15.2699 Heatmap Loss - 0.0020 Offset Loss - 1.2897 Size Loss - 0.5224 Heading Loss -
--	---	---

Loss - 0.0107 Epoch 3 [899/1080]: Loss - 6.4736 Heatmap Loss - 0.0012 Offset Loss - 0.5830 Size Loss - 0.3356 Heading Loss - 0.0019 Epoch 3 [999/1080]: Loss - 11.6190 Heatmap Loss - 0.0020 Offset Loss - 0.9902 Size Loss - 0.4592 Heading Loss - 0.0106 1080it [06:47, 2.65it/s] Epoch 4 [99/1080]: Loss - 9.2431 Heatmap Loss - 0.0012 Offset Loss - 0.8525 Size Loss - 0.2780 Heading Loss - 0.0032 Epoch 4 [199/1080]: Loss - 6.0708 Heatmap Loss - 0.0013 Offset Loss - 0.5493 Size Loss - 0.2134 Heading Loss - 0.0023 Epoch 4 [299/1080]: Loss - 13.8719 Heatmap Loss - 0.0021 Offset Loss - 1.1608 Size Loss - 0.5565 Heading Loss - 0.0150 Epoch 4 [399/1080]: Loss - 9.4517 Heatmap Loss - 0.0016 Offset Loss - 0.7839 Size Loss - 0.2227 Heading Loss - 0.0123 Epoch 4 [499/1080]: Loss - 4.8003 Heatmap Loss - 0.0014 Offset Loss - 0.4076 Size Loss - 0.2989 Heading Loss - 0.0028 Epoch 4 [599/1080]: Loss - 10.0130 Heatmap Loss - 0.0014 Offset Loss - 0.8876 Size Loss - 0.3650 Heading Loss - 0.0063 Epoch 4 [699/1080]: Loss - 10.1106 Heatmap Loss - 0.0015 Offset Loss - 0.8160 Size Loss - 0.3482 Heading Loss - 0.0145 Epoch 4 [799/1080]: Loss - 8.7612 Heatmap Loss - 0.0014 Offset Loss - 0.7297 Size Loss - 0.3100 Heading Loss - 0.0102 Epoch 4 [899/1080]: Loss - 7.1353 Heatmap Loss - 0.0018 Offset Loss - 0.6195 Size Loss - 0.2980 Heading Loss - 0.0046 Epoch 4 [999/1080]: Loss - 10.6994 Heatmap Loss - 0.0016 Offset Loss - 0.8843 Size Loss - 0.3437 Heading Loss - 0.0135 960it [02:41, 5.95it/s]	Loss - 0.0143 Epoch 3 [899/1080]: Loss - 6.5226 Heatmap Loss - 0.0013 Offset Loss - 0.5933 Size Loss - 0.2777 Heading Loss - 0.0018 Epoch 3 [999/1080]: Loss - 11.7437 Heatmap Loss - 0.0026 Offset Loss - 0.9807 Size Loss - 0.4507 Heading Loss - 0.0123 1080it [07:12, 2.50it/s] Epoch 4 [99/1080]: Loss - 8.9582 Heatmap Loss - 0.0015 Offset Loss - 0.8024 Size Loss - 0.2881 Heading Loss - 0.0049 Epoch 4 [199/1080]: Loss - 4.8636 Heatmap Loss - 0.0015 Offset Loss - 0.4282 Size Loss - 0.2609 Heading Loss - 0.0017 Epoch 4 [299/1080]: Loss - 11.4307 Heatmap Loss - 0.0032 Offset Loss - 0.9447 Size Loss - 0.6611 Heading Loss - 0.0101 Epoch 4 [399/1080]: Loss - 9.5838 Heatmap Loss - 0.0021 Offset Loss - 0.7956 Size Loss - 0.3171 Heading Loss - 0.0110 Epoch 4 [499/1080]: Loss - 4.7270 Heatmap Loss - 0.0018 Offset Loss - 0.4121 Size Loss - 0.2352 Heading Loss - 0.0019 Epoch 4 [599/1080]: Loss - 9.2517 Heatmap Loss - 0.0021 Offset Loss - 0.7996 Size Loss - 0.4799 Heading Loss - 0.0057 Epoch 4 [699/1080]: Loss - 10.1556 Heatmap Loss - 0.0022 Offset Loss - 0.7533 Size Loss - 0.3323 Heading Loss - 0.0207 Epoch 4 [799/1080]: Loss - 7.5240 Heatmap Loss - 0.0017 Offset Loss - 0.6212 Size Loss - 0.3492 Heading Loss - 0.0079 Epoch 4 [899/1080]: Loss - 7.6431 Heatmap Loss - 0.0024 Offset Loss - 0.6578 Size Loss - 0.2587 Heading Loss - 0.0057 Epoch 4 [999/1080]: Loss - 12.2759 Heatmap Loss - 0.0020 Offset Loss - 0.9676 Size Loss - 0.5237 Heading Loss - 0.0188 960it [02:49, 5.66it/s]	0.0165 Epoch 3 [899/1080]: Loss - 6.5790 Heatmap Loss - 0.0010 Offset Loss - 0.5899 Size Loss - 0.3616 Heading Loss - 0.0022 Epoch 3 [999/1080]: Loss - 11.7516 Heatmap Loss - 0.0016 Offset Loss - 0.9897 Size Loss - 0.4866 Heading Loss - 0.0121 1080it [07:11, 2.50it/s] Epoch 4 [99/1080]: Loss - 9.6191 Heatmap Loss - 0.0010 Offset Loss - 0.8930 Size Loss - 0.2717 Heading Loss - 0.0032 Epoch 4 [199/1080]: Loss - 5.2228 Heatmap Loss - 0.0011 Offset Loss - 0.4632 Size Loss - 0.2500 Heading Loss - 0.0023 Epoch 4 [299/1080]: Loss - 11.2540 Heatmap Loss - 0.0017 Offset Loss - 0.9416 Size Loss - 0.4970 Heading Loss - 0.0117 Epoch 4 [399/1080]: Loss - 9.1650 Heatmap Loss - 0.0014 Offset Loss - 0.7755 Size Loss - 0.3079 Heading Loss - 0.0097 Epoch 4 [499/1080]: Loss - 4.8725 Heatmap Loss - 0.0013 Offset Loss - 0.4187 Size Loss - 0.3030 Heading Loss - 0.0026 Epoch 4 [599/1080]: Loss - 8.3244 Heatmap Loss - 0.0012 Offset Loss - 0.7094 Size Loss - 0.3971 Heading Loss - 0.0071 Epoch 4 [699/1080]: Loss - 8.9821 Heatmap Loss - 0.0013 Offset Loss - 0.6774 Size Loss - 0.3900 Heading Loss - 0.0169 Epoch 4 [799/1080]: Loss - 7.8115 Heatmap Loss - 0.0011 Offset Loss - 0.6438 Size Loss - 0.2579 Heading Loss - 0.0100 Epoch 4 [899/1080]: Loss - 7.4767 Heatmap Loss - 0.0015 Offset Loss - 0.6475 Size Loss - 0.3143 Heading Loss - 0.0053 Epoch 4 [999/1080]: Loss - 11.5360 Heatmap Loss - 0.0013 Offset Loss - 0.9377 Size Loss - 0.4315 Heading Loss - 0.0159 960it [02:48, 5.71it/s]
--	--	--

Training loss using Part A:

```
((csc490) wolf:~/csc490/3D-Object-Detection-For-Self-Driving-Vehicles$ srun -p cs]
[c490-compute -c 8 -N 1 --gres gpu python3 -m detection.main train --data_root="/]
[u/csc490h/dataset" --output_root=/h/u3/c8/04/yuxiangn/csc490/results      ]
1080it [06:50, 2.63it/s]
Epoch 0 [99/1080]: Loss - 66.5528 Heatmap Loss - 0.0088 Offset Loss - 4.1087 Siz
e Loss - 17.6367 Heading Loss - 0.0695
Epoch 0 [199/1080]: Loss - 63.5132 Heatmap Loss - 0.0064 Offset Loss - 3.7514 Si
ze Loss - 3.8241 Heading Loss - 0.2154
Epoch 0 [299/1080]: Loss - 57.5492 Heatmap Loss - 0.0064 Offset Loss - 3.7330 Si
ze Loss - 2.7787 Heading Loss - 0.1680
Epoch 0 [399/1080]: Loss - 39.9444 Heatmap Loss - 0.0074 Offset Loss - 2.8427 Si
ze Loss - 1.0873 Heading Loss - 0.0969
Epoch 0 [499/1080]: Loss - 38.3796 Heatmap Loss - 0.0051 Offset Loss - 2.7180 Si
ze Loss - 1.5604 Heading Loss - 0.0913
Epoch 0 [599/1080]: Loss - 33.6480 Heatmap Loss - 0.0049 Offset Loss - 2.6851 Si
ze Loss - 0.7609 Heading Loss - 0.0554
Epoch 0 [699/1080]: Loss - 30.5211 Heatmap Loss - 0.0051 Offset Loss - 2.1368 Si
ze Loss - 0.7560 Heading Loss - 0.0788
Epoch 0 [799/1080]: Loss - 19.9486 Heatmap Loss - 0.0047 Offset Loss - 1.6253 Si
ze Loss - 0.6729 Heading Loss - 0.0256
Epoch 0 [899/1080]: Loss - 22.6483 Heatmap Loss - 0.0046 Offset Loss - 1.0000 Si
ze Loss - 0.6095 Heading Loss - 0.0498
Epoch 0 [999/1080]: Loss - 15.0710 Heatmap Loss - 0.0000 Offset Loss - 1.0000 Si
ze Loss - 0.0000 Heading Loss - 0.0000
```

CSC490_A1.pdf

```

ze Loss - 0.6095 Heading Loss - 0.0498
Epoch 0 [999/1080]: Loss - 15.2740 Heatmap Loss - 0.0022 Offset Loss - 1.2068 Si
ze Loss - 0.3585 Heading Loss - 0.0262
1080it [06:43, 2.68it/s]
Epoch 1 [99/1080]: Loss - 23.7113 Heatmap Loss - 0.0028 Offset Loss - 1.7705 Siz
e Loss - 0.6706 Heading Loss - 0.0505
Epoch 1 [199/1080]: Loss - 11.3494 Heatmap Loss - 0.0032 Offset Loss - 0.9832 Si
ze Loss - 0.5218 Heading Loss - 0.0068
Epoch 1 [299/1080]: Loss - 18.1277 Heatmap Loss - 0.0021 Offset Loss - 1.3617 Si
ze Loss - 0.7136 Heading Loss - 0.0359
Epoch 1 [399/1080]: Loss - 22.6901 Heatmap Loss - 0.0056 Offset Loss - 1.9293 Si
ze Loss - 0.5330 Heading Loss - 0.0231
Epoch 1 [499/1080]: Loss - 20.9288 Heatmap Loss - 0.0027 Offset Loss - 1.7146 Si
ze Loss - 1.2418 Heading Loss - 0.0227
Epoch 1 [599/1080]: Loss - 22.4314 Heatmap Loss - 0.0044 Offset Loss - 1.7803 Si
ze Loss - 0.4235 Heading Loss - 0.0377
Epoch 1 [699/1080]: Loss - 21.8631 Heatmap Loss - 0.0033 Offset Loss - 1.8464 Si
ze Loss - 0.6108 Heading Loss - 0.0246
Epoch 1 [799/1080]: Loss - 17.8667 Heatmap Loss - 0.0043 Offset Loss - 1.4172 Si
ze Loss - 0.5616 Heading Loss - 0.0270
Epoch 1 [899/1080]: Loss - 13.6176 Heatmap Loss - 0.0021 Offset Loss - 1.2188 Si
ze Loss - 0.4708 Heading Loss - 0.0075
Epoch 1 [999/1080]: Loss - 18.0241 Heatmap Loss - 0.0034 Offset Loss - 1.4448 Si
ze Loss - 0.4816 Heading Loss - 0.0275
1080it [06:52, 2.62it/s]
Epoch 2 [99/1080]: Loss - 10.6111 Heatmap Loss - 0.0015 Offset Loss - 0.9827 Siz
e Loss - 0.3705 Heading Loss - 0.0026
Epoch 2 [199/1080]: Loss - 14.9791 Heatmap Loss - 0.0018 Offset Loss - 1.1501 Si
ze Loss - 0.8757 Heading Loss - 0.0242
Epoch 2 [299/1080]: Loss - 21.1717 Heatmap Loss - 0.0049 Offset Loss - 1.6829 Si
ze Loss - 0.6393 Heading Loss - 0.0322
Epoch 2 [399/1080]: Loss - 10.6836 Heatmap Loss - 0.0018 Offset Loss - 0.8924 Si
ze Loss - 0.3437 Heading Loss - 0.0124
Epoch 2 [499/1080]: Loss - 10.5372 Heatmap Loss - 0.0023 Offset Loss - 0.9299 Si
ze Loss - 0.4728 Heading Loss - 0.0054
Epoch 2 [599/1080]: Loss - 11.9931 Heatmap Loss - 0.0036 Offset Loss - 1.0281 Si
ze Loss - 0.3226 Heading Loss - 0.0103
Epoch 2 [699/1080]: Loss - 13.2153 Heatmap Loss - 0.0047 Offset Loss - 1.1315 Si
ze Loss - 0.4266 Heading Loss - 0.0100
Epoch 2 [799/1080]: Loss - 11.3486 Heatmap Loss - 0.0020 Offset Loss - 0.9364 Si
ze Loss - 0.4979 Heading Loss - 0.0129
Epoch 2 [899/1080]: Loss - 6.3589 Heatmap Loss - 0.0010 Offset Loss - 0.5574 Siz
e Loss - 0.3302 Heading Loss - 0.0035
Epoch 2 [999/1080]: Loss - 6.5060 Heatmap Loss - 0.0014 Offset Loss - 0.5509 Siz
e Loss - 0.4254 Heading Loss - 0.0043
1080it [06:42, 2.68it/s]
Epoch 3 [99/1080]: Loss - 6.5771 Heatmap Loss - 0.0016 Offset Loss - 0.5856 Size
Loss - 0.2378 Heading Loss - 0.0032
Epoch 3 [199/1080]: Loss - 7.3647 Heatmap Loss - 0.0020 Offset Loss - 0.6078 Siz
e Loss - 0.3230 Heading Loss - 0.0076
Epoch 3 [299/1080]: Loss - 12.6152 Heatmap Loss - 0.0027 Offset Loss - 0.9880 Si
ze Loss - 0.5712 Heading Loss - 0.0190
Epoch 3 [399/1080]: Loss - 11.5708 Heatmap Loss - 0.0026 Offset Loss - 0.9560 Si
ze Loss - 0.4538 Heading Loss - 0.0129

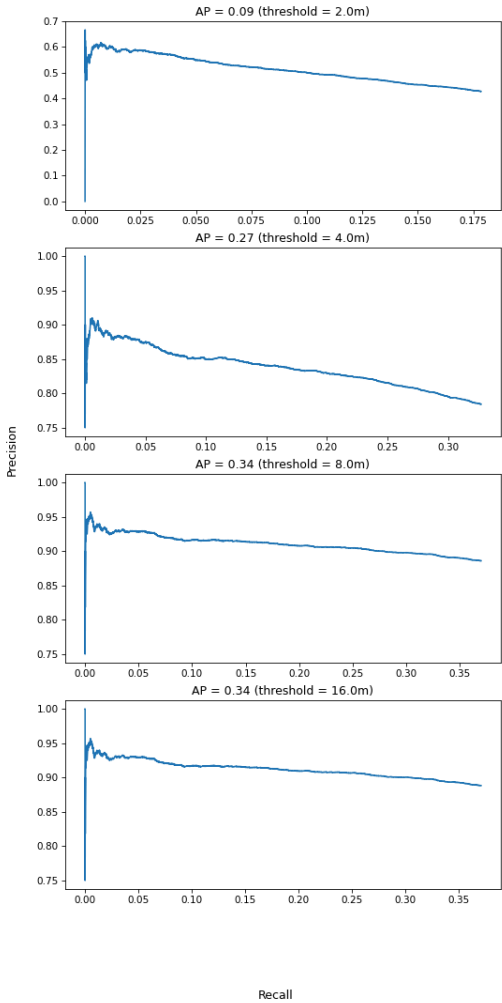
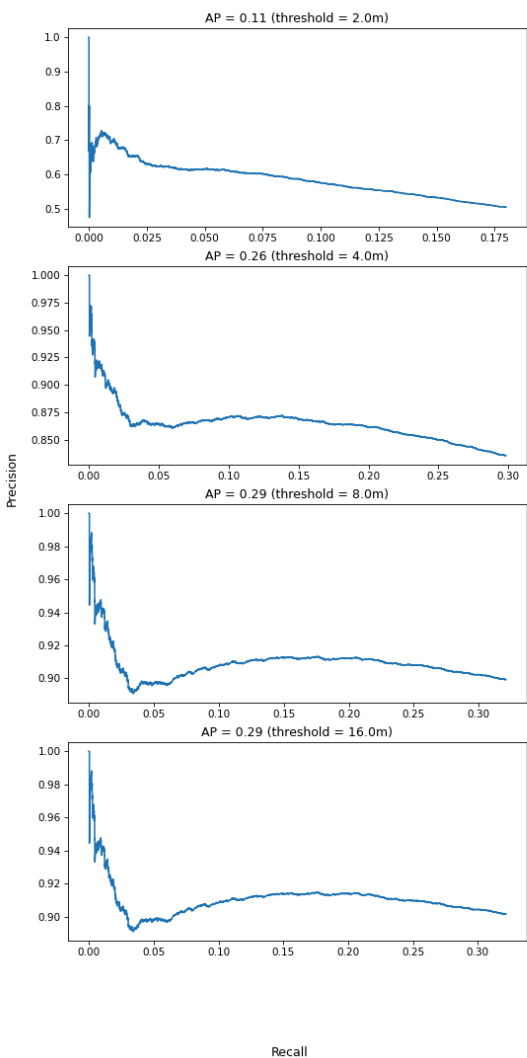
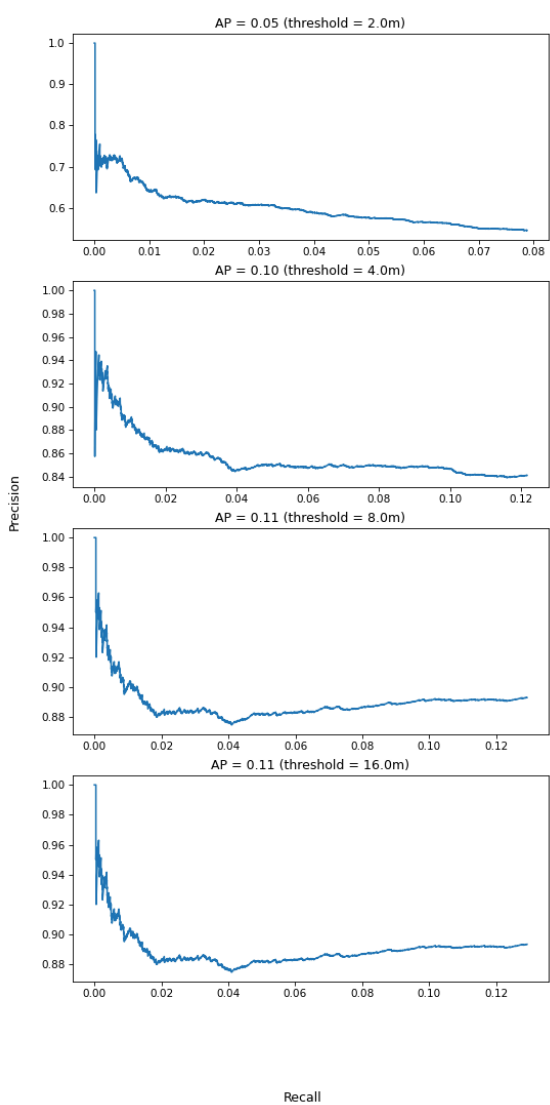
```

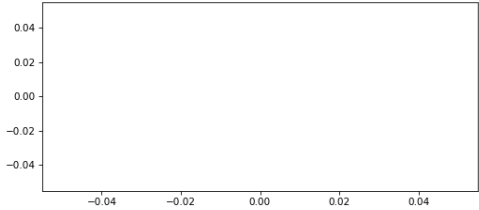
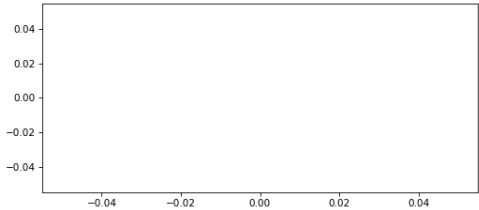
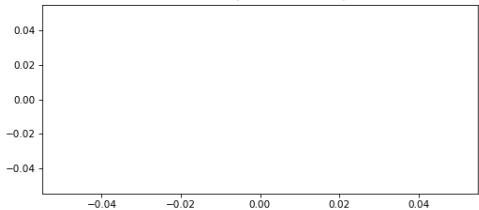
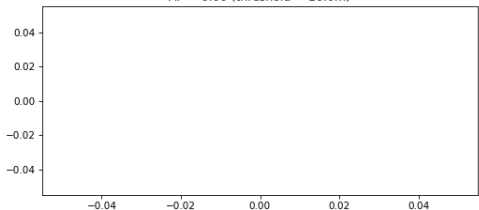
```
e Loss - 0.2745 Heading Loss - 0.0054
Epoch 3 [799/1080]: Loss - 14.8487 Heatmap Loss - 0.0036 Offset Loss - 1.2574 Si
ze Loss - 0.5518 Heading Loss - 0.0136
Epoch 3 [899/1080]: Loss - 6.2452 Heatmap Loss - 0.0014 Offset Loss - 0.5553 Siz
e Loss - 0.2990 Heading Loss - 0.0025
Epoch 3 [999/1080]: Loss - 12.6502 Heatmap Loss - 0.0023 Offset Loss - 1.0552 Si
ze Loss - 0.4753 Heading Loss - 0.0139
1080it [06:48, 2.64it/s]
Epoch 4 [99/1080]: Loss - 10.5528 Heatmap Loss - 0.0016 Offset Loss - 0.9730 Siz
e Loss - 0.2592 Heading Loss - 0.0040
Epoch 4 [199/1080]: Loss - 5.2701 Heatmap Loss - 0.0013 Offset Loss - 0.4716 Siz
e Loss - 0.2172 Heading Loss - 0.0021
Epoch 4 [299/1080]: Loss - 13.3701 Heatmap Loss - 0.0026 Offset Loss - 1.1385 Si
ze Loss - 0.4159 Heading Loss - 0.0131
Epoch 4 [399/1080]: Loss - 10.0434 Heatmap Loss - 0.0020 Offset Loss - 0.8126 Si
ze Loss - 0.3237 Heading Loss - 0.0139
Epoch 4 [499/1080]: Loss - 4.9625 Heatmap Loss - 0.0013 Offset Loss - 0.4190 Siz
e Loss - 0.3597 Heading Loss - 0.0029
Epoch 4 [599/1080]: Loss - 10.3233 Heatmap Loss - 0.0020 Offset Loss - 0.8818 Si
ze Loss - 0.3942 Heading Loss - 0.0091
Epoch 4 [699/1080]: Loss - 10.8134 Heatmap Loss - 0.0019 Offset Loss - 0.4190 Siz
e Loss - 0.2688 Heading Loss - 0.0172
Epoch 4 [799/1080]: Loss - 8.5325 Heatmap Loss - 0.0017 Offset Loss - 0.6740 Siz
e Loss - 0.4800 Heading Loss - 0.0113
```

Trash

```
Epoch 4 [899/1080]: Loss - 6.5859 Heatmap Loss - 0.0020 Offset Loss - 0.5640 Siz
e Loss - 0.2669 Heading Loss - 0.0048
Epoch 4 [999/1080]: Loss - 11.3218 Heatmap Loss - 0.0020 Offset Loss - 0.8952 Si
ze Loss - 0.3481 Heading Loss - 0.0182
960it [02:40, 5.98it/s]
/h/u3/c8/04/yuxiangn/csc490/3D-Object-Detection-For-Self-Driving-Vehicles/detect
ion/metrics/average_precision.py:91: UserWarning: To copy construct from a tenso
r, it is recommended to use sourceTensor.clone().detach() or sourceTensor.clone(
).detach().requires_grad_(True), rather than torch.tensor(sourceTensor).
  label_j_distances = distances[:, j]
      2.0      4.0      8.0     16.0     mean
0 0.006656 0.025665 0.03083 0.03099 0.023535
```

Trash

Evaluate	Focal loss	Rotated Anisotropic Gaussian	Focal loss & rotated Anisotropic Gaussian
	2, 0.25		
PRcurve	Precision-Recall Curves  AP = 0.09 (threshold = 2.0m) AP = 0.27 (threshold = 4.0m) AP = 0.34 (threshold = 8.0m) AP = 0.34 (threshold = 16.0m)	Precision-Recall Curves  AP = 0.11 (threshold = 2.0m) AP = 0.26 (threshold = 4.0m) AP = 0.29 (threshold = 8.0m) AP = 0.29 (threshold = 16.0m)	Precision-Recall Curves  AP = 0.05 (threshold = 2.0m) AP = 0.10 (threshold = 4.0m) AP = 0.11 (threshold = 8.0m) AP = 0.11 (threshold = 16.0m)
	1, 0.25		

	3, 0.25		
	0.5,0.5 (loss become nan in the later epochs)		
	Precision-Recall Curves AP = 0.00 (threshold = 2.0m)  AP = 0.00 (threshold = 4.0m)  AP = 0.00 (threshold = 8.0m)  AP = 0.00 (threshold = 16.0m)  Precision Recall		

Show compelling quantitative results

Experiment 1 (Just focal loss)

Alpha	Gamma	Average Precision	Average Precision (% change)
NAN (Part A AP)	NAN (Part A AP)	0.45	0%
.25	2	0.34	-24.4%
.25	1	0.27	-40%
.25	3	0.40	-11.1%
.10	3	0.25	-44.4%
.20	3	0.32	-28.9%

The table here shows the average precision at a threshold = 16, for all the (alpha, gamma) pairs we have tested. All show a decrease in the average precision when compared to the original code that we implemented in Part A. Among them, however, the (0.25, 3) pair gives us the best result.

Experiment 2 (Just Anisotropic Gaussian Kernel)

Scale X	Scale Y	Average Precision	Average Precision (% change)
NAN (Part A AP)	NAN (Part A AP)	0.45	0%
$2 * (\text{boxsize_x}^2) / \text{heatmap_norm_scale}$	$2 * (\text{boxsize_y}^2) / \text{heatmap_norm_scale}$	0.29	-35.6%
$1 * (\text{boxsize_x}^2) / \text{heatmap_norm_scale}$	$1 * (\text{boxsize_y}^2) / \text{heatmap_norm_scale}$	0.13	-71.1%

The table here shows the average precision at threshold = 16, for all the scale_x, scale_y combinations we have tried. All show a decrease in the average precision when compared to the original code that we implemented in Part A. Among them, the $2 * (\text{boxsize_x})^2$, $2(\text{boxsize_y})^2$ gives us the best result.

Limitation

Are there setups in which your approach fails

- We're allowing the model to take more risk in making the predictions when using focal loss in training. We see that the decrease in scores for the detections in each frame illustrates that the model is less confident about its predictions, as a trade-off to actually making the prediction of vehicles existing at the pixel locations.
- When combining the focal loss technique with the rotated anisotropic gaussian technique using the separately tuned hyperparameters didn't improve the average precision results. The focal loss only approach was able to achieve average precision of 0.40, but when combining with anisotropic gaussian technique, the average precision was only 0.18 for AP threshold of 16.

Are there potential paths of future work to improve your approach

Automated Focal Loss

Tuning hyperparameters alpha and gamma in the focal loss calculation, for addressing the class imbalance problem, makes the training process more complicated. Michael Weber et al [4] proposed the automated focal loss which uses an adaptive gamma. Since gamma is the hyperparameter that determines the importance of negative misclassified examples, the adaptive gamma approach enables us to use a higher gamma value in earlier stage of training to place more focus on the hard examples (misclassified), while employing smaller gammas (towards 0 value) as we progress in training to preserve the weighting of having true negative and true positive predictions.

Dual Cross-entropy Focal Loss

Using the Dual Cross-entropy Focal Loss proposed by Xuanlin He et al. [5], we can potentially improve the performance of our vehicle detection model. In addition to trying to increase the probability of correctly classified predictions and focusing on the hard examples as in the focal loss, the Dual version also promotes decreasing the probability of misclassified predictions. As a result, they were able to accelerate the training process with the Dual Cross-entropy Focal Loss.

sample x_k as follows:

$$\begin{aligned} DCFL(x_k, \mathbf{Y}_k) = & - \left(1 - p_k^{(t_k)}\right)^{\gamma_1} \cdot \log\left(p_k^{(t_k)}\right) \\ & + \beta \cdot \sum_{\substack{i=1 \\ i \neq t_k}}^C \left(\left(p_k^{(i)}\right)^{\gamma_2} \cdot \log\left(\alpha + p_k^{(i)}\right) \right) \end{aligned} \quad (8)$$

$\gamma_1 \geq 0, \gamma_2 \geq 0$. When $\gamma_1 = \gamma_2 = 0$, the dual cross-entropy focal loss is the same as the dual

Focal Loss Ratio

Building on top of focal loss, Maikel Kerkhof et al. [6] showed in their paper that using their Focal Loss Ratio may be better than the original focal loss for training, because Focal Loss Ratio penalizes the hard (false negative) examples even more so than focal loss.

$$FLR(y, \hat{y}) = \frac{-\alpha(1 - \hat{y})^\gamma CE(y, \hat{y})}{\frac{1}{n} \sum_{i=1}^n -\alpha(1 - \hat{y})^\gamma CE(y_s, \hat{y})}, \quad (6)$$

where y are the true labels, y_s are the shuffled labels, CE is the categorical cross-entropy, and n is the number of sample-incorrect label pairs to use. In Equation 6, α and γ are

Other thoughts for improving our approach

- In busier intersections, interchanges or traffic jams where the traffic is very dense, the class imbalance might not be too much of a problem because the vehicles would occupy a large proportion of the total number of pixels.
- We can incorporate offset values into creating the predicted heatmaps by combining predicted centroid and offset to be the center for the gaussian kernel.
- For vehicles that are farther from our self-driving vehicle, we can try assigning smaller weights so we place less focus on those farther vehicles. Detecting vehicles that are far away is of less importance in our motion planning.
- With inspiration from recurrent neural networks' memory units, we can potentially utilize the RADAR sensor's vehicle velocity information to aid the creation of predicted heatmap for the next frame (i.e. next time step) in the sequence. By making use of longer-term dependencies, we can define a region that the vehicle is likely to be in the next frame and then increase the weight of any pixel from the gaussian kernel that lies within this region, for constructing the predicted heatmap in the next frame.

References

1. https://openaccess.thecvf.com/content_ICCV_2017/papers/Lin_Focal_Loss_for_ICCV_2017_paper.pdf
2. <https://towardsdatascience.com/class-imbalance-random-sampling-and-data-augmentation-with-imbalanced-learn-63f3a92ef04a>
3. <https://arxiv.org/pdf/1905.07290v1.pdf>
4. <https://arxiv.org/pdf/1904.09048.pdf>
5. https://link.springer.com/chapter/10.1007/978-3-030-61401-0_11
6. <https://eprint.iacr.org/2021/1408.pdf>